

A
INDUSTRIAL ORIENTED MINI PROJECT REPORT
ON
SMART VEHICLE SAFETY & ALERT SYSTEM
Submitted to partial fulfillment of the requirements for the award of the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

By
CHEBOLU SURYA PRASHANTHI **23911A6685**
PANDYALA KUSHAL BABU **23911A66B3**
SETTY SIRI **23911A66B8**
MYLARAM AKSHATH SAI **24915A6612**

Under the Guidance of
Mr. Srinu Naik
Assistant Professor



Department of Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

VIDYA JYOTHI INSTITUTE OF TECHNOLOGY
(An Autonomous Institution)

(Approved by AICTE, Accredited by NAAC, NBA & permanently affiliated to JNTUH, Hyderabad)
Aziz Nagar Gate, C.B. Post, Hyderabad-500075

2025-2026



VIDYA JYOTHI

INSTITUTE OF TECHNOLOGY

AN AUTONOMOUS INSTITUTION

(Approved by AICTE, Accredited by NAAC, NBA & permanently Affiliated to JNTUHI, Hyderabad)

Aziz Nagar Gate, C.B. Post, Hyderabad-500075

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

CERTIFICATE

This is to certify that the project report titled "**SMART VEHICLE SAFETY & ALERT SYSTEM**" is being submitted by **CHEBOLU SURYA PRASHANTHI (23911A6685), PANDYALA KUSHAL BABU (23911A66B3), SETTY SIRI (23911A66B8), MYLARAM AKSHATH SAI (24915A6612)** in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning), is a record of bonafide work carried out by them under my guidance and supervision. These results embodied in this project report have not been submitted to any other University or Institute for the award of any degree or diploma.

Internal Guide

Mr. Srinu Naik
Assistant Professor

Head of Department

Dr. N. Anusha
Associate Professor

External Examiner

DECLARATION

We, **CHEBOLU SURYA PRASHANTI, PANDYALA. KUSHAL BABU, SETTY SIRI, and MYLARAM AKSHATH SAI**, hereby declare that the project entitled “**SMART VEHICLE SAFETY & ALERT SYSTEM**”, submitted in partial fulfillment of the requirements for the Bachelor of Technology degree in Computer Science and Engineering(Artificial Intelligence and Machine Learning), is our original work. This project has not been previously submitted for the award of any degree or qualification at any other institution.

CHEBOLU SURYA PRASHANTHI	23911A6685
PANDYALA KUSHAL BABU	23911A66B3
SETTY SIRI	23911A66B8
MYLARAM AKSHATH SAI	24915A6612

Date:

Place:

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to our project guide, **Mr. Srinu Naik**, Assistant Professor at Vidya Jyothi Institute of Technology, Hyderabad, for his invaluable support, guidance, and insights throughout the course of this project. His encouragement and expertise have been instrumental in helping us deepen our understanding and enhance our skills, and we are truly grateful for his mentorship.

Our sincere thanks also go to **Dr. N.Anusha**, Associate Professor and Head of the Department of Computer Science and Engineering(Artificial Intelligence and Machine Learning), for her unwavering support and motivation during our academic journey. Her dedication to academic excellence has inspired us, and her assistance has been vital to the success of our work.

We are especially grateful to our Principal, **Dr. A.Srujana**, for providing the necessary infrastructure and resources that made this project possible. Her commitment to fostering a supportive learning environment has enabled us to fully explore and develop our ideas.

Lastly, we extend our deep appreciation to our parents for their constant encouragement, as well as to the dedicated faculty members who have guided us throughout the course. Their contributions have greatly shaped our progress and have been instrumental in bringing us to this stage in our academic journey.

CHEBOLU SURYA PRASHANTHI	23911A6685
PANDYALA KUSHAL BABU	23911A66B3
SETTY SIRI	23911A66B8

ABSTRACT

SMART VEHICLE SAFETY & ALERT SYSTEM

The increasing number of road accidents caused by driver negligence, alcohol consumption, and delayed emergency response has become a major concern worldwide. This project presents a Vehicle Safety System designed using the Arduino UNO, integrated with GSM and GPS modules, to enhance driver safety and enable real-time monitoring.

The system incorporates an alcohol sensor to detect the presence of alcohol in the driver's breath. If alcohol is detected beyond a predefined threshold, the system restricts vehicle ignition and activates a buzzer alert to warn the driver. Additionally, a push button is included as an emergency trigger, allowing the user to manually send distress signals in case of accidents or critical situations.

The integrated GPS module continuously tracks the vehicle's location, while the GSM module sends real-time alert messages containing location coordinates to predefined emergency contacts. A red LED (RLED) provides visual indication of unsafe conditions such as alcohol detection or emergency activation.

This system aims to reduce accidents, ensure quick emergency response, and improve overall road safety through automation and real-time communication. The proposed solution is cost-effective, reliable, and suitable for implementation in modern smart vehicles.

INDEX

S.NO	TITLE	PAGE NO.
1	Introduction	1-7
1.1	Introduction	1-7
1.1.1	Overview of Vehicle Safety Systems	1-2
1.1.2	Need for Advanced Safety System	2
1.1.3	Proposed Vehicle Safety System	2-3
1.1.4	Advantages of Using Embedded Systems	3
1.1.5	Applications and Future Scope	3-4
1.2	Existing System	4-5
1.2.1	Features of Existing System	4
1.2.2	Limitations over Existing Systems	4-5
1.3	Proposed System	5-7
1.3.1	Working of Proposed System	5-6
1.3.2	Features of Proposed System	6
1.3.3	Advantages Over Existing system	6
1.3.4	Block Diagram	7
2.	Literature Survey	8-11
2.1	Web Scrapping Techniques and Tools	8
2.1.1	Alcohol Detection Based Vehicle Control Systems	8
2.1.2	GPS and GSM Based Vehicle Tracking Systems	8-9
2.1.3	Accident Detection and Alert Systems	9-10
2.1.4	IoT-Based Smart Vehicle Safety Systems	10
2.1.5	Emergency Button-Based Alert Systems	10
2.1.6.	Identified Research Gap	10
2.1.7.	Motivation for Proposed System	10-11

3.	Description of Hardware Components	12-43
3.1	General Description of ARDUINO	12
3.1.1	General Description of ARDUINO	12
3.1.2	Need for ARDUINO	13
3.1.3	Types of ARDUINO Boards	14
3.1.4	Elements of ARDUINO Boards	14-15
3.1.5	Why ARDUINO?	16-17
3.1.6	ARDUINO Features	17-18
3.1.7	Pin Description	19-24
3.2.	Power Supply	24-29
3.3	Circuit Diagram	30
3.4	Buzzer	30-33
3.5	Alcohol Sensor	34-35
3.6	GPS(Global Positioning System)	36-39
3.7	GSM Model	39-44
4.	Circuit Diagram	45-49
4.1	Existing System	45
4.2	Source Code	45-50
4.3	Flowchart	51
5.	Software Development	52-66
5.1	Software Development	52-66
6.	Output Images	66-67
7.	Conclusion	68
8.	References	69

LIST OF FIGURES

S. NO	TITLE	PAGE NO.
1.	Block design proposal for the system	7
2.	Pin Description	19
3.	Power supply	24
4.	Transformer circuit	25
5.	Have wave rectifier circuit	26
6.	Full wave rectifier circuit	26
7.	Bridge rectifier circuit	27
8.	Bridge rectifier circuit-2	27
9.	Filter capacitor	28
10.	Voltage reactor circuit	29
11.	Circuit diagram of power supply	30
12.	Buzzer	31
13.	Buzzer - 2	32
14.	Buzzer circuit	33
15.	Alcohol Sensor	34
16.	Configuration	35
17.	Technical data circuit	35
18.	GSM	40
19.	Structure of GSM network	40
20.	Working of GSM	41
21.	Circuit diagram	45
22.	Flowchart	51
23.	Arduino board	52
24.	Arduino configuration	54
25.	Drive configuration	55
26.	Arduino board connection	56
27.	Uploading program from file	56
28.	Code for Arduino	57
29.	Applying tools	58

30.	Adding examples	59
31.	Arduino console - 1	60
32.	Arduino console - 2	61
33.	Arduino console - 3	62
34.	Arduino Motherboard	62
35.	Arduino file selection	64
36.	Arduino console - 4	65
37.	Before the alert	66
38.	After the alert	67

CHAPTER - 1

INTRODUCTION

1.1. Introduction

1.1.1 Overview of Vehicle Safety Systems

Road transportation plays a crucial role in modern society, enabling the movement of people and goods efficiently. However, with the rapid increase in the number of vehicles, road safety has become a major global concern. Accidents caused by driver negligence, alcohol consumption, fatigue, and lack of timely medical assistance continue to result in significant loss of life and property.

To address these issues, vehicle safety systems have evolved from basic mechanical features to advanced electronic and intelligent systems. Traditional safety measures such as seat belts and airbags help reduce the severity of accidents, but they do not prevent accidents from occurring. Therefore, there is a growing need for active safety systems that can detect risky conditions and take preventive actions.

In this context, embedded systems and IoT technologies are playing a transformative role. Microcontrollers like the Arduino UNO enable real-time monitoring, decision-making, and communication, making them ideal for developing smart vehicle safety solutions. By integrating sensors, communication modules, and alert systems, it is possible to design a system that not only detects unsafe conditions but also responds immediately.

1.1.2. Need for an Advanced Safety System

One of the leading causes of road accidents is drunk driving, which significantly impairs a driver's reaction time, coordination, and judgment. Despite strict laws and awareness campaigns, alcohol-related accidents remain high. This highlights the need for automated systems that can detect alcohol consumption and prevent the vehicle from operating.

Another major issue is the delay in emergency response after accidents. In many cases, victims do not receive timely medical assistance due to lack of communication or inability to share their location. This delay can be life-threatening, especially in remote areas.

To overcome these challenges, modern safety systems integrate Global Positioning System (GPS) and Global System for Mobile Communication (GSM) technologies. GPS helps in determining the exact location of the vehicle, while GSM enables communication with emergency contacts or authorities. When combined, these technologies can automatically send alerts along with location details in case of emergencies.

Additionally, providing a manual emergency trigger, such as a push button, allows the user to send distress signals instantly. This feature is particularly useful in situations where the driver senses danger or requires immediate help.

Thus, there is a clear need for a comprehensive vehicle safety system that includes:

- Alcohol detection
- Real-time location tracking
- Emergency alert communication
- Audible and visual warning systems

1.1.3. Proposed Vehicle Safety System

The proposed system is a smart vehicle safety solution that integrates multiple components to enhance driver and passenger safety. The core of the system is the Arduino UNO, which acts as the central processing unit and coordinates all operations.

An alcohol sensor is used to monitor the driver's breath. If alcohol is detected above a predefined threshold, the system triggers a buzzer alert and activates a red LED (RLED) to indicate danger. This ensures that the driver is immediately warned, and preventive action can be taken.

The system also includes a push button that acts as an emergency switch. When pressed, it sends an alert message through the GSM module to predefined contacts, such as family members or emergency services.

The GPS module continuously tracks the vehicle's location, and this information is transmitted via the GSM module in the form of SMS. This enables real-time tracking and quick response during emergencies.

Key features of the proposed system include:

- Detection of alcohol consumption
- Immediate alert through buzzer and LED
- Real-time GPS tracking
- GSM-based SMS alerts
- Manual emergency triggering

This system is designed to be **cost-effective, easy to implement, and highly reliable**, making it suitable for a wide range of vehicles.

1.1.4. Advantages of Using Embedded Systems

Embedded systems offer several advantages in vehicle safety applications:

- **Real-time operation:** Immediate detection and response to unsafe conditions
- **Low cost:** Affordable components like Arduino and sensors
- **Compact design:** Easy integration into vehicles
- **Scalability:** Can be expanded with additional features such as IoT or cloud monitoring
- **Reliability:** Stable performance in various environments

The use of Arduino simplifies development due to its open-source nature, availability of libraries, and ease of programming.

1.1.5. Applications and Future Scope

The proposed vehicle safety system has wide applications, including:

- Personal vehicles
- Commercial transport systems
- School buses and public transport

- Fleet management systems

In the future, this system can be enhanced by integrating:

- IoT-based real-time monitoring dashboards
- Mobile applications for alerts and tracking
- AI-based driver behavior analysis
- Automatic accident detection using additional sensors

Such advancements can further improve safety and reduce the number of road accidents significantly.

In conclusion, the increasing rate of road accidents demands innovative and intelligent safety solutions. The integration of alcohol detection, GPS tracking, and GSM communication into a single system provides a comprehensive approach to vehicle safety. The proposed system not only helps in preventing accidents but also ensures timely assistance during emergencies, making it a valuable contribution to modern transportation safety.

1.2 Existing System

In current vehicle safety approaches, most systems are either **manual** or **partially automated**, focusing mainly on post-accident protection rather than prevention. Traditional safety mechanisms include seat belts, airbags, and basic alarm systems, which help reduce the severity of accidents but do not actively prevent them.

1.2.1. Features of Existing Systems

- Use of **seat belts and airbags** for passenger protection
- Basic **vehicle alarms** for theft prevention
- Manual emergency communication (mobile phone calls)
- Limited or no real-time monitoring of driver condition

1.2.2. Limitations of Existing Systems

1. No Alcohol Detection Mechanism

Existing systems do not automatically detect whether the driver is under the influence of alcohol. This leads to a high number of accidents caused by drunk driving.

2. Lack of Real-Time Tracking

Most vehicles do not have built-in GPS tracking for emergency situations. Even if available, it is not always linked to alert systems.

3. Delayed Emergency Response

In case of accidents, the victim must manually call for help. If the driver is unconscious, no alert is sent.

4. No Integrated Communication System

There is no automatic system to send location-based alerts to emergency contacts using mobile networks.

5. No Immediate Warning System

Existing systems lack instant alerts like buzzer or LED indicators to warn about unsafe conditions.

6. Dependence on Human Action

Most safety responses rely on human intervention, which may not be possible during critical situations.

1.3. Proposed System

To overcome the limitations of existing systems, the proposed solution introduces a smart vehicle safety system using the Arduino UNO integrated with GSM and GPS technologies.

This system is designed to provide real-time monitoring, automatic alerts, and preventive safety measures.

Key Components of Proposed System

- Arduino UNO (controller)
- Alcohol Sensor
- GSM Module
- GPS Module
- Push Button (Emergency switch)
- Buzzer
- Red LED (RLED)

1.3.1. Working of Proposed System

1. The alcohol sensor continuously monitors the driver's breath.
2. If alcohol is detected above a threshold:
 - The buzzer is activated
 - The red LED glows as a warning
 - (Optional) Vehicle ignition can be disabled

3. The GPS module tracks the real-time location of the vehicle.
4. The GSM module sends SMS alerts with location details to predefined contacts.
5. The push button allows manual emergency alerts in case of accidents or danger.

1.3.2. Features of Proposed System

- **Alcohol Detection and Prevention**
Prevents drunk driving by detecting alcohol levels.
- **Real-Time GPS Tracking**
Provides accurate location data during emergencies.
- **Automatic SMS Alerts via GSM**
Sends alerts with coordinates to emergency contacts.
- **Manual Emergency Trigger**
Push button enables immediate distress signaling.
- **Audio-Visual Alerts**
Buzzer and LED provide instant warning signals.
- **Low Cost and Easy Implementation**
Uses affordable and widely available components.

1.3.3. Advantages Over Existing System

Existing System	Proposed System
No alcohol detection	Detects alcohol in real time
No automatic alerts	Sends SMS alerts automatically
No location tracking	GPS-based location tracking
Manual emergency response	Automatic + manual emergency system
Limited safety features	Integrated smart safety system
Reactive (after accident)	Preventive + responsive

1.3.4. Block Design for the System

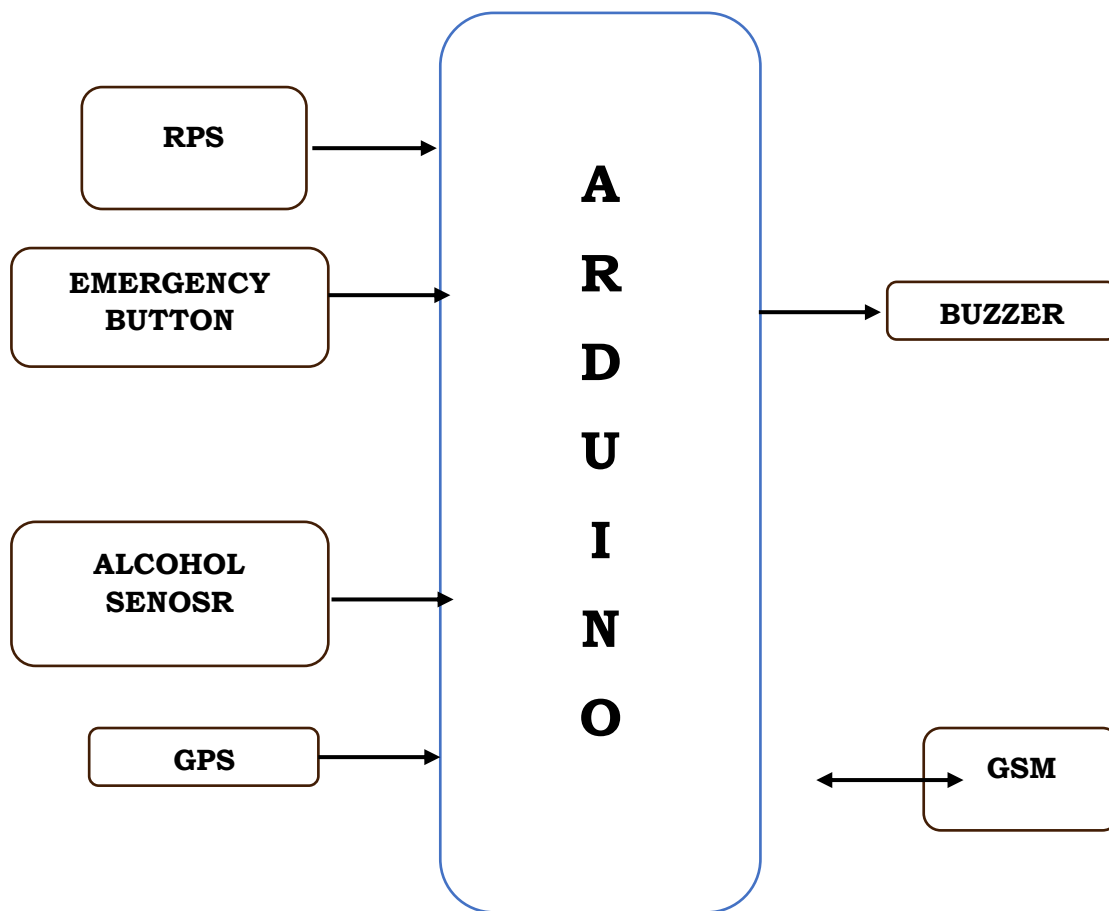


Fig 1.1 Block Design Proposal for the System

CHAPTER 2

LITERATURE SURVEY

2.1 Web Scraping Techniques and Tools

The development of vehicle safety systems has gained significant attention in recent years due to the increasing number of road accidents. Researchers and engineers have proposed various solutions using embedded systems, IoT, and communication technologies to improve safety and reduce accident risks. This section reviews some of the important existing works related to vehicle safety, alcohol detection, and emergency alert systems.

2.1.1. Alcohol Detection Based Vehicle Control Systems

Several studies have focused on detecting alcohol consumption to prevent drunk driving. These systems typically use **MQ-series alcohol sensors** interfaced with microcontrollers like the Arduino UNO.

- The sensor detects alcohol concentration in the driver's breath.
- If the level exceeds a predefined threshold, the system disables the vehicle ignition.

Limitations Identified

- Most systems only stop the vehicle but do not send alerts.
- Lack of integration with communication modules like GSM.
- No tracking or monitoring capabilities.

2.1.2. GPS and GSM Based Vehicle Tracking Systems

Many researchers have proposed vehicle tracking systems using **GPS and GSM modules** to monitor vehicle location in real time.

- GPS module provides latitude and longitude coordinates.
- GSM module sends SMS alerts containing location details.

Applications

- Fleet management

- Theft tracking
- Emergency response systems

Limitations Identified

- Focus mainly on tracking, not accident prevention.

No integration with driver condition monitoring (like alcohol detection).

2.1.3. Accident Detection and Alert Systems

Some advanced systems use sensors such as accelerometers and vibration sensors to detect accidents automatically.

- Sudden impact or abnormal motion triggers an alert.
- GSM module sends emergency messages with GPS location.

Limitations Identified

- Detection occurs only after an accident happens.
- False triggering may occur due to road conditions.

No preventive measures included

2.1.4. IoT-Based Smart Vehicle Safety Systems

With the advancement of IoT, modern systems integrate cloud platforms, mobile applications, and real-time monitoring dashboards.

- Data is uploaded to cloud servers for remote monitoring.
- Users can track vehicle status via mobile apps.

Advantages

- Real-time monitoring
- Data storage and analysis

Limitations Identified

- Higher cost and complexity
- Requires internet connectivity

- Not always suitable for low-cost implementations

2.1.5. Emergency Button-Based Alert Systems

Some systems include a **manual emergency button** that allows users to send distress signals.

- When pressed, an alert message is sent via GSM.
- Useful in medical emergencies or dangerous situations.

Limitations Identified

- Depends on user action
- Ineffective if the driver is unconscious

2.1.6. Identified Research Gap

From the above studies, the following gaps are identified:

- Lack of integrated systems combining prevention and response
- Absence of alcohol detection with real-time alert communication
- Limited use of both automatic and manual emergency features together
- Need for a low-cost, reliable, and easy-to-implement solution

2.1.7. Motivation for Proposed System

The proposed system addresses these gaps by combining multiple features into a single platform:

- Alcohol detection to prevent accidents
- GPS tracking for real-time location monitoring
- GSM communication for instant alerts
- Push button for manual emergency activation

- Buzzer and LED for immediate warning

This integrated approach ensures both preventive and responsive safety, making the system more effective compared to existing solutions.

CHAPTER 3

DESCRIPTION OF HARDWARE COMPONENTS

3.1 General Description of ARDUINO

3.1.1. General Description of ARDUINO

Arduino is an open source microcontroller which can be easily programmed, erased and reprogrammed at any instant of time. Introduced in 2005 the Arduino platform was designed to provide an inexpensive and easy way for hobbyists, students and professionals to create devices that interact with their environment using sensors and actuators. Based on simple microcontroller boards, it is an open source computing platform that is used for constructing and programming electronic devices. It is also capable of acting as a mini computer just like other microcontrollers by taking inputs and controlling the outputs for a variety of electronics devices.

It is also capable of receiving and sending information over the internet with the help of various Arduino shields, which are discussed in this paper. Arduino uses a hardware known as the Arduino development board and software for developing the code known as the Arduino IDE (Integrated Development Environment). Built up with the 8-bit Atmel AVR microcontroller's that are manufactured by Atmel or a 32-bit Atmel ARM, these microcontrollers can be programmed easily using the C or C++ language in the Arduino IDE. Unlike the other microcontroller boards in India, the Arduino boards entered the electronic market only a couple of years ago, and were restricted to small scale projects only. People associated with electronics are now gradually coming up and accepting the role of Arduino for their own projects.

This development board can also be used to burn (upload) a new code to the board by simply using a USB cable to upload. The Arduino IDE provides a simplified integrated platform which can run on regular personal computers and allows users to write programs for Arduino using C or C++. With so many Arduino boards available in the market, selecting a particular development board needs a variety of survey done with respect to their specifications and capabilities, which can be used for the project execution according to its specified applications.

3.1.2 Need for ARDUINO

Why is there a need to use Arduino in specific? or What makes it different from others? Massimo Banzi, a Co-founder of Arduino mentions some very important reasons for this question.

1) Active User Community:

A group of people using a similar product can hold posted message conversations and share their experiences or solve the problems of the other users in the communities with their own experiences [1]. “If you start charging for everything, everything dies very quickly.” says Banzi, Arduino Cofounder.

2) Growth of Arduino:

Arduino was developed with intent to provide an economical and trouble-free way for hobbyists, students and professionals to build devices that interact with their situation using sensors and actuators. This makes it perfect for newcomers to get started quickly [1].

3) Inexpensive Hardware:

Since Arduino is an open source platform the software is not purchased and only the cost of buying the board or its parts is incurred, thus making it very cheap. The hardware designs are also available online for free from its official website [1].

4) Arduino Board as a Programmer:

To make Arduino board function easy and also making it available everywhere these boards come with a USB cable for power requirements as well as functioning as a programmer [1]. 5) Multi-platform Environment: The Arduino IDE is capable of running on a number of platforms including Microsoft, Linux and Mac OS X making the user community even larger [1].

3.1.3. Type of ARDUINO Boards

Arduino boards are available with many different types of built-in modules in it. Boards such as

Arduino BT come with a built-in Bluetooth module, for wireless communication. These built-in modules can also be available separately which can then be interfaced (mounted) to it. These modules are known as Shield. Some of the most commonly used Shields are:

Arduino Ethernet shield: It that allows an Arduino board to connect to the internet using the Ethernet library and to read and write an SD card using the SD library [2].

Arduino Wireless shield: It allows your Arduino board to communicate wirelessly using Zigbee [2].

Arduino Motor Driver Shield: It allows your Arduino boards to interface with driver of a motor etc. [2].

3.4. Elements of ARDUINO Boards

Elements of an Arduino Board can be done into two categories:

- Hardware
- Software

Hardware The Arduino Development Board consists of many components that together makes it work. Here are some of those main component blocks that help in its functioning:

Microcontroller: This is the heart of the development board, which works as a mini computer and can receive as well as send information or command to the peripheral devices connected to it. The microcontroller used differs from board to board; it also has its own various specifications.

External Power Supply: This power supply is used to power the Arduino development board with a regulated voltage ranging from 9 – 12 volts.

USB plug: This plug is a very important port in this board. It is used to upload (burn) a program to the microcontroller using a USB cable. It also has a regulated power of 5V which also powers the Arduino board in cases when the External Power Supply is absent.

Internal Programmer: The developed software code can be uploaded to the microcontroller via USB port, without an external programmer.

Reset button: This button is present on the board and can be used to reset the Arduino microcontroller.

Analog Pins: There are some analog input pins ranging from A0 – A7 (typical). These pins are used for the analog input / output. The no. of analog pins also varies from board to board.

Digital I/O Pins: There are some digital input pins also ranging from 2 to 16 (typical). These pins are used for the digital input / output. The no. of these digital pins also varies from board to board.

Power and GND Pins: There are pins on the development board that provide 3.3, 5 volts and ground through them.

Software

The program code written for Arduino is known as a sketch. The software used for developing such sketches for an Arduino is commonly known as the Arduino IDE. This IDE contains the following parts in it:

Text editor: This is where the simplified code can be written using a simplified version of C++ programming language.

Message area: It displays error and also gives a feedback on saving and exporting the code.

Text: The console displays text output by the Arduino environment including complete error messages and other information

Console Toolbar: This toolbar contains various buttons like Verify, Upload, New, Open, Save and Serial Monitor. On the bottom right hand

3.1.5 Why ARDUINO?

Thanks to its simple and accessible user experience, Arduino has been used in thousands of different projects and applications. The Arduino software is easy-to-use for beginners, yet flexible enough for advanced users. It runs on Mac, Windows, and Linux. Teachers and students use it to build low cost scientific instruments, to prove chemistry and physics principles, or to get started with programming and robotics. Designers and architects build interactive prototypes, musicians and artists use it for installations and to experiment with new musical instruments. Makers, of course, use it to build many of the projects exhibited at the Maker Faire, for example. Arduino is a key tool to learn new things. Anyone - children, hobbyists, artists, programmers - can start tinkering just following the step by step instructions of a kit, or sharing ideas online with other members of the Arduino community.

There are many other microcontrollers and microcontroller platforms available for physical computing. Parallax Basic Stamp, Netmedia's BX-24, Phidgets, MIT's Handyboard, and many others offer similar functionality. All of these tools take the messy details of microcontroller programming and wrap it up in an easy-to-use package. Arduino also simplifies the process of working with microcontrollers, but it offers some advantage for teachers, students, and interested amateurs over other systems:

- **Inexpensive** - Arduino boards are relatively inexpensive compared to other microcontroller platforms. The least expensive version of the Arduino module can be assembled by hand, and even the pre-assembled Arduino modules cost less than \$50
- **Cross-platform** - The Arduino Software (IDE) runs on Windows, Macintosh OSX, and Linux operating systems. Most microcontroller systems are limited to Windows.
- **Simple, clear programming environment** - The Arduino Software (IDE) is easy-to-use for beginners, yet flexible enough for advanced users to take advantage of as well. For teachers, it's conveniently based on the Processing programming environment, so students learning to program in that environment will be familiar with how the Arduino IDE works.
- **Open source and extensible software** - The Arduino software is published as open source tools, available for extension by experienced programmers. The language can be expanded through C++ libraries, and people wanting to understand the technical details can make the leap from Arduino to the AVR C programming language on

which it's based. Similarly, you can add AVR-C code directly into your Arduino programs if you want to.

- **Open source and extensible hardware** - The plans of the Arduino boards are published under a Creative Commons license, so experienced circuit designers can make their own version of the module, extending it and improving it. Even relatively inexperienced users can build the breadboard version of the module in order to understand how it works and save money.

3.1.6 ARDUINO Features

High performance, low power AVR® 8-bit microcontroller

- ✓ Advanced RISC architecture
- ✓ 131 powerful instructions – most single clock cycle execution 8 general purpose working registers × • 32
- ✓ Fully static operation
- ✓ Up to 16MIPS throughput at 16MHz
- ✓ On-chip 2-cycle multiplier
- ✓ High endurance non-volatile memory segments
- ✓ 32K bytes of in-system self-programmable flash program memory
- ✓ 1Kbytes EEPROM
- ✓ 2Kbytes internal SRAM
- ✓ Write/erase cycles: 10,000 flash/100,000 EEPROM
- ✓ Optional boot code section with independent lock bits
- ✓ In-system programming by on-chip boot program
- ✓ True read-while-write operation
- ✓ Programming lock for software security
- ✓ Peripheral features

- ✓ Two 8-bit Timer/Counters with separate prescaler and compare mode
- ✓ One 16-bit Timer/Counter with separate prescaler, compare mode, and capture mode
- ✓ Real time counter with separate oscillator
- ✓ Six PWM channels
- ✓ 8-channel 10-bit ADC in TQFP and QFN/MLF package
- ✓ Temperature measurement
- ✓ Programmable serial USART
- ✓ Master/slave SPI serial interface
- ✓ Byte-oriented 2-wire serial interface (Phillips I2 C compatible)
- ✓ Programmable watchdog timer with separate on-chip oscillator
- ✓ On-chip analog comparator
- ✓ Interrupt and wake-up on pin change
- ✓ Special microcontroller features
- ✓ Power-on reset and programmable brown-out detection
- ✓ Internal calibrated oscillator
- ✓ External and internal interrupt sources
- ✓ Six sleep modes: Idle, ADC noise reduction, power-save, power-down, standby, and extended standby

3.1.7 Pin Description

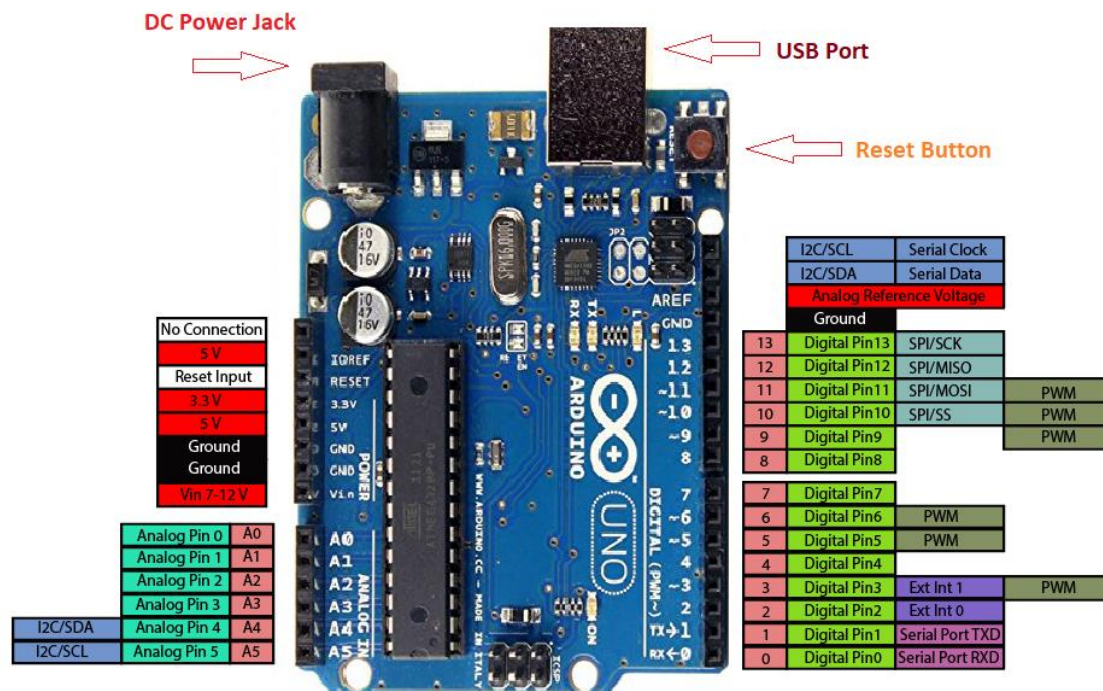


Fig 3.1 Pin Description

Arduino Uno pinout - Power Supply

There are 3 ways to power the Arduino Uno:

- **Barrel Jack** - The Barrel jack, or DC Power Jack can be used to power your Arduino board. The barrel jack is usually connected to a wall adapter. The board can be powered by 5-20 volts but the manufacturer recommends to keep it between 7-12 volts. Above 12 volts, the regulators might overheat, and below 7 volts, might not suffice.
- **VIN Pin** - This pin is used to power the Arduino Uno board using an external power source. The voltage should be within the range mentioned above.
- **USB cable** - when connected to the computer, provides 5 volts at 500mA.

There is a polarity protection diode connecting between the positive of the barrel jack to the VIN pin, rated at 1 Ampere.

The power source you use determines the power you have available for your circuit. For instance, powering the circuit using the USB limits you to 500mA. Take into consideration that this is also used for powering the MCU, its peripherals, the on-board regulators, and the components connected to it. When powering your circuit through the barrel jack or VIN, the maximum capacity available is determined by the 5 and 3.3 volts regulators on-board the Arduino.

- **5v and 3v3**

They provide regulated 5 and 3.3v to power external components according to manufacturer specifications.

- **GND**

In the Arduino Uno pinout, you can find 5 GND pins, which are all interconnected.

The GND pins are used to close the electrical circuit and provide a common logic reference level throughout your circuit. Always make sure that all GNDs (of the Arduino, peripherals and components) are connected to one another and have a common ground.

- **RESET** - resets the Arduino
- **IOREF** - This pin is the input/output reference. It provides the voltage reference with which the microcontroller operates.

Arduino Uno Pinout - Analog IN

The Arduino Uno has 6 **analog pins**, which utilize ADC (Analog to Digital converter).

These pins serve as analog inputs but can also function as digital inputs or digital outputs.

Analog to Digital Conversion

ADC stands for Analog to Digital Converter. ADC is an electronic circuit used to convert analog signals into digital signals. This digital representation of analog signals allows the processor (which is a digital device) to measure the analog signal and use it through its operation. Arduino Pins A0-A5 are capable of reading analog voltages. On Arduino the ADC

has 10-bit resolution, meaning it can represent analog voltage by 1,024 digital levels. The ADC converts voltage into bits which the microprocessor can understand.

One common example of an ADC is Voice over IP (VoIP). Every smartphone has a microphone that converts sound waves (voice) into analog voltage. This goes through the device's ADC, gets converted into digital data, which is transmitted to the receiving side over the internet.

Arduino Uno Pinout - Digital Pins

Pins 0-13 of the Arduino Uno serve as digital input/output pins.

Pin 13 of the Arduino Uno is connected to the built-in LED.

In the Arduino Uno - pins 3,5,6,9,10,11 have PWM capability.

It's important to note that:

Each pin can provide/sink up to 40 mA max. But the recommended current is 20 mA.

The absolute max current provided (or sank) from all pins together is 200mA

What does digital mean?

Digital is a way of representing voltage in 1 bit: either 0 or 1. Digital pins on the Arduino are pins designed to be configured as inputs or outputs according to the needs of the user. Digital pins are either on or off. When ON they are in a HIGH voltage state of 5V and when OFF they are in a LOW voltage state of 0V.

On the Arduino, When the digital pins are configured as **output**, they are set to 0 or 5 volts.

When the digital pins are configured as **input**, the voltage is supplied from an external device. This voltage can vary between 0-5 volts which is converted into digital representation (0 or 1). To determine this, there are 2 thresholds:

- Below 0.8v - considered as 0.
- Above 2v - considered as 1.

When connecting a component to a digital pin, make sure that the logic levels match. If the voltage is in between the thresholds, the returning value will be undefined.

What is PWM?

In general, Pulse Width Modulation (PWM) is a modulation technique used to encode a message into a pulsing signal. A PWM is comprised of two key components: **frequency** and **duty cycle**. The PWM frequency dictates how long it takes to complete a single cycle (period) and how quickly the signal fluctuates from high to low. The duty cycle determines how long a signal stays high out of the total period. Duty cycle is represented in percentage.

In Arduino, the PWM enabled pins produce a constant frequency of $\sim 500\text{Hz}$, while the duty cycle changes according to the parameters set by the user. See the following illustration:

PWM signals are used for speed control of DC motors, dimming LEDs and more.

Communication Protocols

Serial (TTL) - Digital pins 0 and 1 are the serial pins of the Arduino Uno.

They are used by the onboard USB module.

What is Serial Communication?

Serial communication is used to exchange data between the Arduino board and another serial device such as computers, displays, sensors and more. Each Arduino board has at least one serial port. Serial communication occurs on digital pins 0 (RX) and 1 (TX) as well as via USB. Arduino supports serial communication through digital pins with the SoftwareSerial Library as well. This allows the user to connect multiple serial-enabled devices and leave the main serial port available for the USB.

Software serial and hardware serial - Most microcontrollers have hardware designed to communicate with other serial devices. Software serial ports use a pin-change interrupt system to communicate. There is a built-in library for Software Serial communication. Software serial is used by the processor to simulate extra serial ports. The only drawback with software serial is that it requires more processing and cannot support the same high speeds as hardware serial.

SPI - SS/SCK/MISO/MOSI pins are the dedicated pins for SPI communication. They can be found on digital pins 10-13 of the Arduino Uno and on the ICSP headers.

What is SPI?

Serial Peripheral Interface (SPI) is a serial data protocol used by microcontrollers to communicate with one or more external devices in a bus like connection. The SPI can also be used to connect 2 microcontrollers. On the SPI bus, there is always one device that is denoted as a Master device and all the rest as Slaves. In most cases, the microcontroller is the Master device. The SS (Slave Select) pin determines which device the Master is currently communicating with.

SPI enabled devices always have the following pins:

- MISO (Master In Slave Out) - A line for sending data to the Master device
- MOSI (Master Out Slave In) - The Master line for sending data to peripheral devices
- SCK (Serial Clock) - A clock signal generated by the Master device to synchronize data transmission.

I2C - SCL/SDA pins are the dedicated pins for I2C communication. On the Arduino Uno they are found on Analog pins A4 and A5.

What is I2C?

I2C is a communication protocol commonly referred to as the “I2C bus”. The I2C protocol was designed to enable communication between components on a single circuit board. With I2C there are 2 wires referred to as SCL and SDA.

- SCL is the clock line which is designed to synchronize data transfers.
- SDA is the line used to transmit data.

Each device on the I2C bus has a unique address, up to 255 devices can be connected on the same bus.

Aref - Reference voltage for the analog inputs.

Interrupt - INT0 and INT1. Arduino Uno has two external interrupt pins.

External Interrupt - An external interrupt is a system interrupt that occurs when outside interference is present. Interference can come from the user or other hardware devices in the network. Common uses for these interrupts in Arduino are reading the frequency a square wave generated by encoders or waking up the processor upon an external event.

Arduino has two forms of interrupt:

- External
- Pin Change

There are two external interrupt pins on the ATmega168/328 called INT0 and INT1. both INT0 and INT1 are mapped to pins 2 and 3. In contrast, Pin Change interrupts can be activated on any of the pins.

Arduino Uno Pinout - ICSP Header

ICSP stands for In-Circuit Serial Programming. The name originated from In-System Programming headers (ISP). Manufacturers like Atmel who work with Arduino have developed their own in-circuit serial programming headers. These pins enable the user to program the Arduino boards' firmware. There are six ICSP pins available on the Arduino board that can be hooked to a programmer device via a programming cable.

3.2 Power Supply

All digital circuits require regulated power supply. In this article we are going to learn how to get a regulated positive supply from the mains supply.

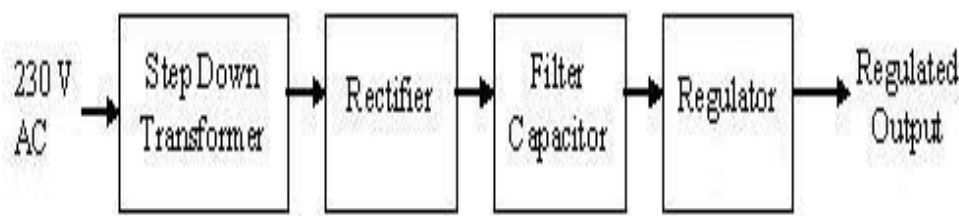


Fig 3.2 Power supply

Figure 2.2 shows the basic block diagram of a fixed regulated power supply. Let us go through each block.

TRANSFORMER

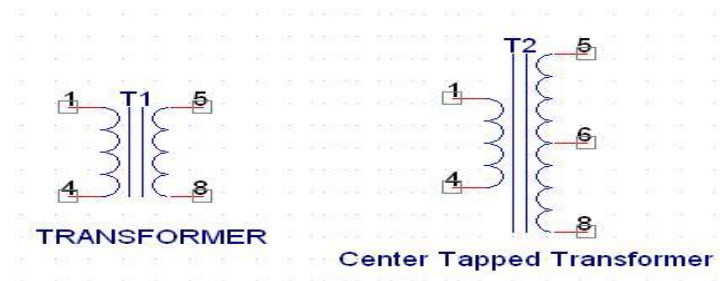


Fig 3.3 Transformer circuit

A transformer consists of two coils also called as “WINDINGS” namely PRIMARY & SECONDARY.

They are linked together through inductively coupled electrical conductors also called as CORE. A changing current in the primary causes a change in the Magnetic Field in the core & this in turn induces an alternating voltage in the secondary coil. If load is applied to the secondary then an alternating current will flow through the load. If we consider an ideal condition then all the energy from the primary circuit will be transferred to the secondary circuit through the magnetic field.

$$P_{\text{primary}} = P_{\text{secondary}}$$

So

Rectifier

A rectifier is a device that converts an AC signal into DC signal. For rectification purpose we use a diode, a diode is a device that allows current to pass only in one direction i.e. when the anode of the diode is positive with respect to the cathode also called as forward biased condition & blocks current in the reversed biased condition.

Rectifier can be classified as follows:

1) Half Wave rectifier

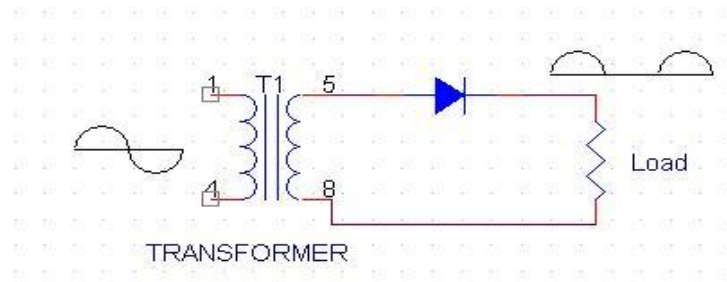


Fig 3.4 Half wave rectifier circuit

This is the simplest type of rectifier as you can see in the diagram a half wave rectifier consists of only one diode. When an AC signal is applied to it during the positive half cycle the diode is forward biased & current flows through it. But during the negative half cycle diode is reverse biased & no current flows through it. Since only one half of the input reaches the output, it is very inefficient to be used in power supplies.

2) Full wave rectifier

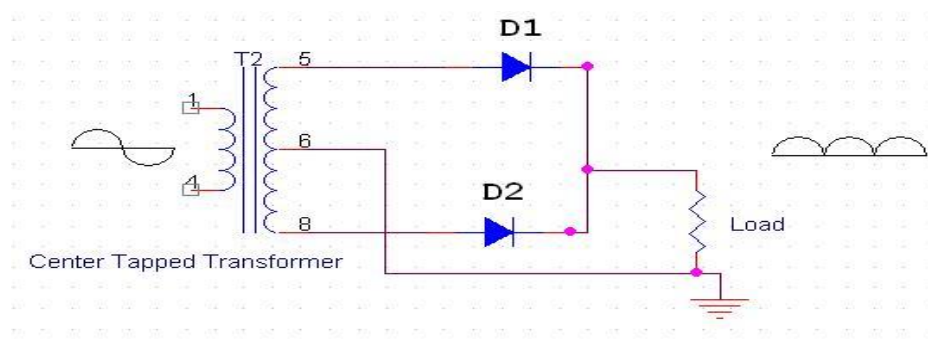


Fig 3.5 Full wave rectifier circuit

Half wave rectifier is quite simple but it is very inefficient, for greater efficiency we would like to use both the half cycles of the AC signal. This can be achieved by using a center tapped transformer i.e. we would have to double the size of secondary winding & provide connection to the center. So during the positive half cycle diode D1 conducts & D2 is in reverse biased condition. During the negative half cycle diode D2 conducts & D1 is reverse biased. Thus we get both the half cycles across the load.

One of the disadvantages of Full Wave Rectifier design is the necessity of using a center tapped transformer, thus increasing the size & cost of the circuit. This can be avoided by using the Full Wave Bridge Rectifier.

3) Bridge Rectifier

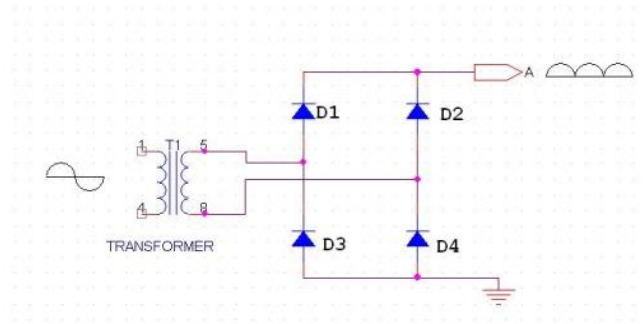


Fig 3.6 Bridge Rectifier circuit

As the name suggests it converts the full wave i.e. both the positive & the negative half cycle into DC thus it is much more efficient than Half Wave Rectifier & that too without using a center tapped transformer thus much more cost effective than Full Wave Rectifier.

Full Bridge Wave Rectifier consists of four diodes namely D1, D2, D3 and D4. During the positive half cycle diodes D1 & D4 conduct whereas in the negative half cycle diodes D2 & D3 conduct thus the diodes keep switching the transformer connections so we get positive half cycles in the output.

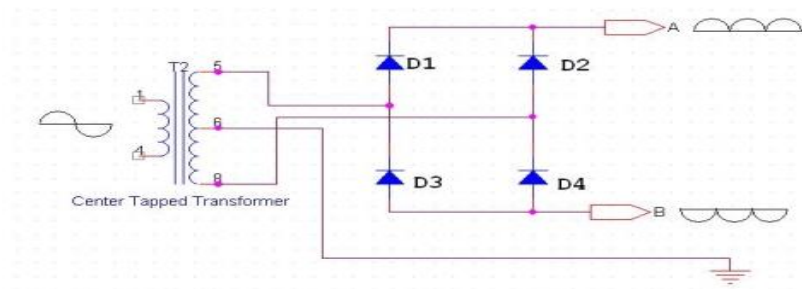


Fig 3.7 Bridge wave rectifier circuit - 2t

If we use a center tapped transformer for a bridge rectifier we can get both positive & negative half cycles which can thus be used for generating fixed positive & fixed negative voltages.

Filter Capacitor

Even though half wave & full wave rectifier give DC output, none of them provides a constant output voltage. For this we require to smoothen the waveform received from the rectifier. This can be done by using a capacitor at the output of the rectifier this capacitor is also called as “FILTER CAPACITOR” or “SMOOTHING CAPACITOR” or “RESERVOIR CAPACITOR”. Even after using this capacitor a small amount of ripple will remain.

We place the Filter Capacitor at the output of the rectifier the capacitor will charge to the peak voltage during each half cycle then will discharge its stored energy slowly through the load while the rectified voltage drops to zero, thus trying to keep the voltage as constant as possible.

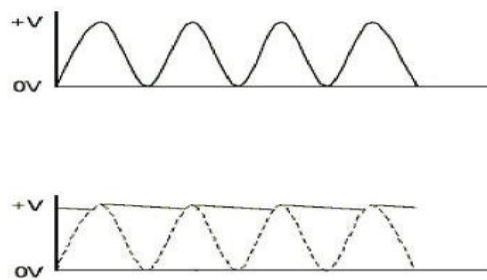


Fig 3.8 Filter Capacitor

If we go on increasing the value of the filter capacitor then the Ripple will decrease. But then the costing will increase. The value of the Filter capacitor depends on the current Consumed by the circuit, the frequency of the waveform & the accepted ripple.

$$C = \frac{V_r F}{I}$$

Where,

V_r = accepted ripple voltage.(should not be more than 10% of the voltage)

I = current consumed by the circuit in Amperes.

F = frequency of the waveform. A half wave rectifier has only one peak in one cycle so $F=25\text{hz}$

Whereas a full wave rectifier has Two peaks in one cycle so $F=100\text{hz}$.

Voltage Regulator

A Voltage regulator is a device which converts varying input voltage into a constant regulated output voltage. Voltage regulator can be of two types

1) Linear Voltage Regulator

Also called as Resistive Voltage regulator because they dissipate the excessive voltage resistively as heat.

2) Switching Regulators.

They regulate the output voltage by switching the Current ON/OFF very rapidly. Since their output is either ON or OFF it dissipates very low power thus achieving higher efficiency as compared to linear voltage regulators. But they are more complex & generate high noise due to their switching action. For low level of output power switching regulators tend to be costly but for higher output wattage they are much cheaper than linear regulators.

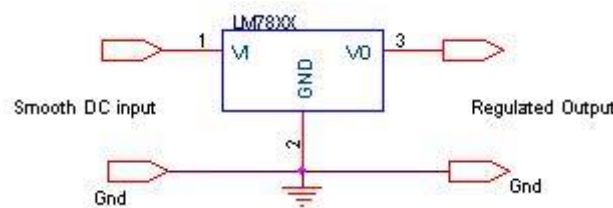


Fig 3.9 Voltage Reactor circuit

After filtering the rectifier output the signal is given to a voltage regulator. The maximum input voltage that can be applied at the input is 35V.

3.3 Circuit diagram

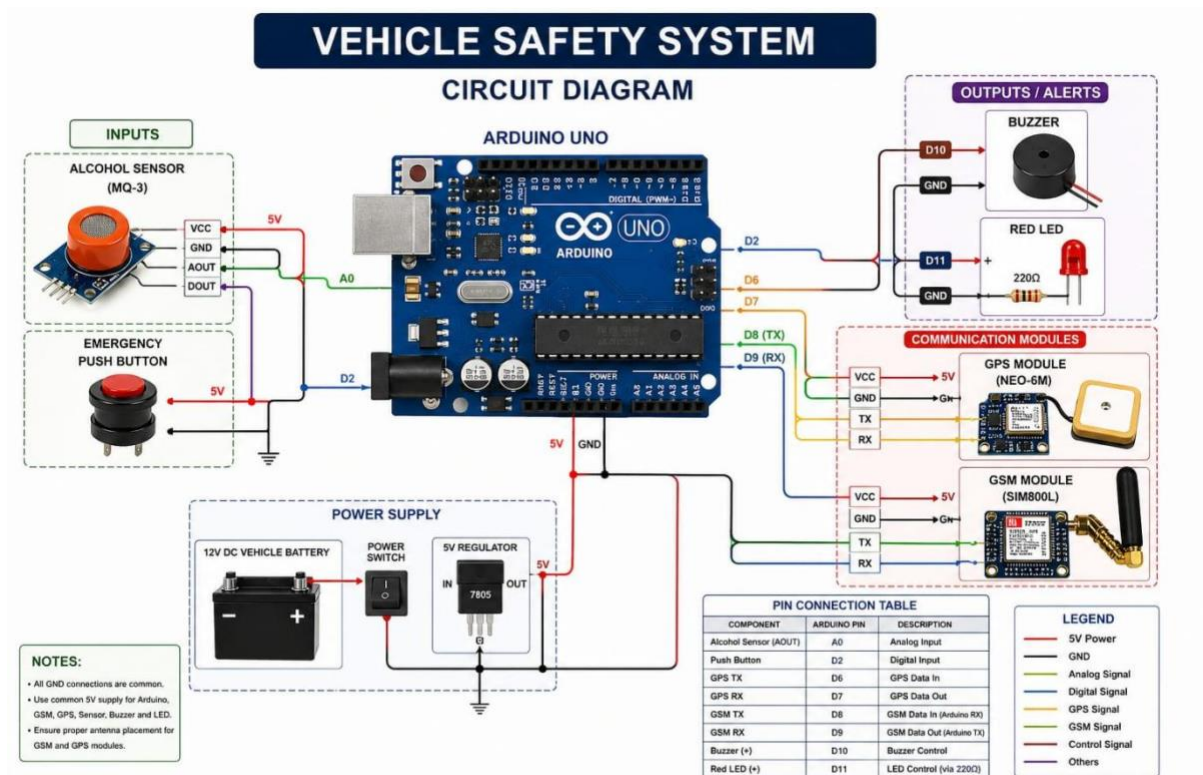


Fig 3.10. Circuit Diagram of power supply

IC 7805:

7805 is an integrated three-terminal positive fixed linear voltage regulator. It supports an input voltage of 10 volts to 35 volts and output voltage of 5 volts. It has a current rating of 1 amp although lower current models are available. Its output voltage is fixed at 5.0V. The 7805 also has a built-in current limiter as a safety feature. 7805 is manufactured by many companies, including National Semiconductors and Fairchild Semiconductors.

3.4 BUZZER

An Active buzzer is also called a piezo buzzer. It is basically **a tiny speaker that you can connect directly to an Arduino**. You can make it sound a tone at a frequency you set. The buzzer produces sound based on reverse of the piezoelectric effect.

A buzzer or piezo speaker is an audio signalling device commonly used to produce sound. Piezo buzzer produces sound based on the reverse principle of the piezoelectric effect. The

buzzer is a less costly and light-weighted electronic device that's why it is used in computers, alarm devices, refrigerators, microwave oven, security devices and so on.

There are two conductors available inside the buzzer along with a piezo crystal between them. Whenever the potential is applied across the crystal then the conductor's position gets changed due to which 2 to 4 kHz sound wave produced by the buzzer.

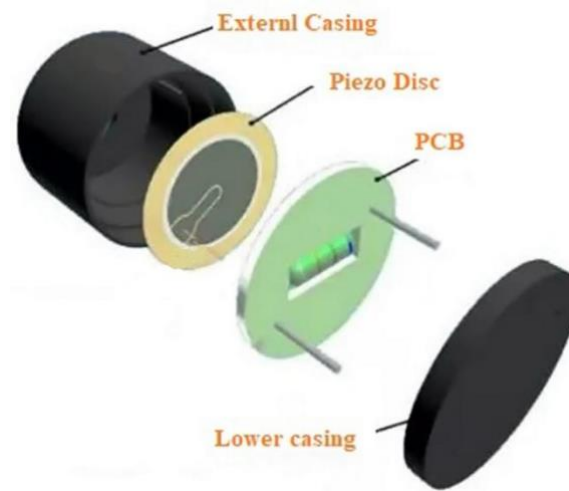


Fig 3.11 Buzzer

If your buzzer has a sticker on top of it, pull the sticker off. Connect one pin (it doesn't matter which one) to the Arduino's ground (Gnd) and the other end to digital pin 10. From the Arduino, you can make sounds with a buzzer by using tone. You have to tell it which pin the buzzer is on, what frequency (in Hertz, Hz) you want, and how long (in milliseconds) you want it to keep making the tone. We will use the **Active buzzer tone generation command**:
`tone(8, 1200, 500); // tone on pin 8 at 1200 Hz for 1/2 second`

How do you know what frequency to use? Young people can generally hear frequencies between about 20 Hz and 20,000 Hz; older people lose the high frequencies and may not be able to hear all the way up to 20,000. Of course, it varies from person to person. Also, your buzzer may not be able to reproduce the whole range, especially the very high and low notes.

If you like, you can leave off the duration. In that case, it will keep making a tone until you tell it to stop by calling `noTone(pin)` or by calling `tone()` with a different frequency.

What is the piezoelectric effect?

Piezoelectric is an effect where certain crystals will change shape when you apply electricity to them. By applying an electric signal at the right frequency, the crystal can make sound.

A buzzer or beeper is an signalling device, which produces sound. It may be mechanical, electromechanical, or piezoelectric.

This piezo buzzer is 23mm in diameter and has 30mm spaced mount holes. Supplied with a 100mm lead it is designed for 3 – 20V, it produces a 3kHz tone at an 85dB level at 30cm.

Specifications:

- OperatioVoltage:3-24VDC
- Current:<15mA
- SPL:85dBA/10cm
- Frequency:3,300Hz
- Color: Black
- Operating Temperature: – 20° to +60°C

Types of buzzers:

Buzzers come in a variety of construction, size, and specifications. Different types and sizes of buzzers are used for different applications. Based on construction, there are the following kinds of buzzers:

- Piezoelectric buzzers.
- Magnetic buzzers.
- Electromagnetic buzzers.
- Mechanical buzzers.
- Electromechanical buzzers.



Fig 3.12 Buzzer - 2

Working of Buzzer:

The main working principle is based on the theory that, whenever an electric potential is applied across a piezoelectric material, a pressure variation is generated. A piezo buzzer consists of piezo crystals in between two conductors.

When a potential difference is applied across these crystals, they push one conductor and pull the other conductor by their internal property. The continuous pull and push action generate a sharp sound wave.

How to use a Buzzer:

A buzzer is a small yet efficient component to add sound features to our project/system. It is very small and compact 2-pin structure hence can be easily used on breadboard, Perf Board and even on PCBs which makes this a widely used component in most electronic applications.

There are two types are buzzers that are commonly available. The one shown here is a simple buzzer which when powered will make a Continuous Beeeeeeppp sound, the other type is called a readymade buzzer which will look bulkier than this and will produce a Beep. Beep. Beep. Sound due to the internal oscillating circuit present inside it. But, the one shown here is most widely used because it can be customized with help of other circuits to fit easily in our application.

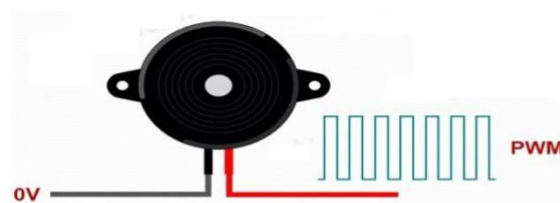


Fig 3.13 Buzzer circuit

This buzzer can be used by simply powering it using a DC power supply ranging from 4V to 9V. A simple 9V battery can also be used, but it is recommended to use a regulated +5V or +6V DC supply. The buzzer is normally associated with a switching circuit to turn ON or turn OFF the buzzer at required time and require interval.

Applications of Buzzer:

- Alarming Circuits, where the user has to be alarmed about something.
- Communication equipment's.
- Automobile electronics.
- Portable equipment's, due to its compact size.
- Piezo buzzers generate a loud & sharp sound. So, they are typically used as an alarm circuit.

3.5 ALCOHOL SENSOR

Sensitive material of MQ-3 gas sensor is SnO_2 , which with lower conductivity in clean air. When the target alcohol gas exists, the sensor's conductivity is higher along with the gas concentration rising. Please use simple electro circuit, convert change of conductivity to correspond output signal of gas concentration.

MQ-3 gas sensor has high senility to Alcohol, and has good resistance to disturb of gasoline, smoke and vapor. The sensor could be used to detect alcohol with different concentration; it is with low cost and suitable for different application.



Fig 3.14 Alcohol Sensor

Configuration

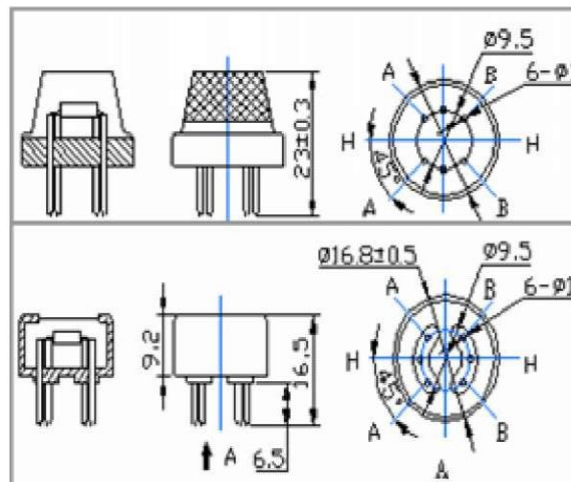


Fig 3.15

Character

- Good sensitivity to alcohol gas
- Long life and low cost
- Simple drive circuit

Application

- Vehicle alcohol detector
- Portable alcohol detector

Technical Data

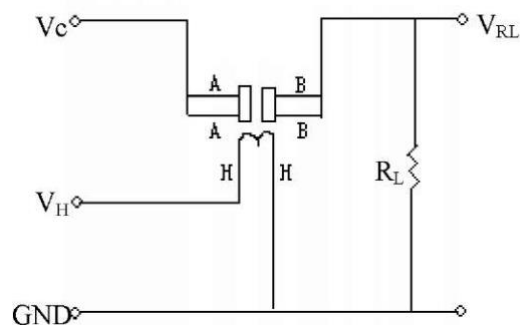


Fig 3.16 Technical Data Circuit

The above is basic test circuit of the sensor. The sensor needs to be put 2 voltage, heater voltage (VH) and test voltage (VC). VH used to supply certified working temperature to the sensor, while VC used to detect voltage (VRL) on load resistance (RL) whom is in series with sensor. The sensor has light polarity, Vc need DC power. VC and VH could use same power circuit with precondition to assure performance of sensor. In order to make the sensor with better performance, suitable RL value is needed: Power of Sensitivity body (Ps)

3.6 GPS (Global Positioning System)

Introduction

The Global Positioning System (GPS) is a U.S. space-based radio navigation system that provides reliable positioning, navigation, and timing services to civilian users on a continuous worldwide basis -- freely available to all. For anyone with a GPS receiver, the system will provide location and time. GPS provides accurate location and time information for an unlimited number of people in all weather, day and night, anywhere in the world.

The GPS is made up of three parts:

1. Satellites orbiting the Earth
2. Control and monitoring stations on Earth
3. The GPS receivers owned by users.

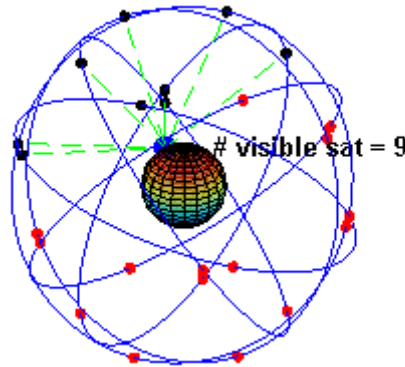
GPS satellites broadcast signals from space that are picked up and identified by GPS receivers. Each GPS receiver then provides three-dimensional location (latitude, longitude, and altitude) plus the time.

1. Components of GPS

a) Space Segment

- Consists of 24+ satellites orbiting Earth at 20,200 km with 55° inclination.
- Each satellite completes two orbits per day.
- Key features:

- Built to last ~10 years, solar-powered with backup batteries.
- Broadcasts L1 (1.57542 GHz) for civilians and L2 (1.2276 GHz) for military.
- Uses CDMA spread-spectrum with C/A and P(Y) codes for timing and identification.
- Orbits arranged so at least 4 satellites are always visible globally.



The GPS satellite system

b) Control Segment

- 5 ground stations: Hawaii, Ascension Island, Diego Garcia, Kwajalein, Colorado Springs.
- These monitor satellite health and synchronize onboard atomic clocks using updates.
- GPS satellites undergo periodic orbital adjustments; maneuvers require temporary deactivation from use.

c) User Segment

- Includes military, commercial, and civil users.
- GPS receivers compute position using signals from at least 4 satellites.
- Advanced receivers use WAAS (accuracy <3 meters) and DGPS (3–5 meters accuracy).
- Multi-channel designs ensure quick satellite locking even in urban or wooded environments.

2. Working Principle

GPS works using **trilateration**, determining distance from satellites based on signal travel time. The intersection of spheres (from 3 satellites) gives a 2D fix, while a 4th satellite adds altitude and precise timing. The high accuracy is achieved through:

- Atomic clock synchronization.
- Correction signals via WAAS or DGPS.
- Error handling for atmospheric distortion, clock errors, and signal multipath.

3. Applications of GPS

- **Navigation:** aviation, marine, vehicle, pedestrian.
- **Emergency services:** location tracking and coordination.
- **Infrastructure:** telecom synchronization, banking transactions.
- **Agriculture & science:** precision farming, geological surveys.
- **Everyday use:** smartphones, mapping, fitness tracking.

4. GPS Commands (NMEA Sentences)

a) GGA - Global Positioning System Fixed Data

- Format: \$GPGGA,time,lat,N/S,long,E/W,fix,sats,HDOP,alt,M,sep,M,,ID*checksum
- Example:
\$GPGGA,053740.000,2503.6319,N,12136.0099,E,1,08,1.1,63.8,M,15.2,M,,0000*64

b) GLL - Geographic Position – Latitude/Longitude

- Format: \$GPGLL,lat,N/S,long,E/W,time,status,mode*checksum
- Example: \$GPGLL,2503.6319,N,12136.0099,E,053740.000,A,A*52

c) GSA - GNSS DOP and Active Satellites

- Format: \$GPGSA,mode1,mode2,SV1...SV12,PDOP,HDOP,VDOP*checksum

- Example: \$GPGSA,A,3,24,07,17,11,28,08,20,04,,,,,2.0,1.1,1.7*35

d) GSV - GNSS Satellites in View

- Format:
\$GPGSV,num_msg,msg_num,sats_in_view,SV_ID,elev,azim,SNR,...*checksum
- Multiple messages used if satellites >4.
- Example:

bash

CopyEdit

```
$GPGSV,3,1,12,28,81,285,42,24,67,302,46,...
```

```
$GPGSV,3,2,12,17,41,328,45,...
```

e) RMC - Recommended Minimum GNSS Data

- Format: \$GPRMC,time,status,lat,N/S,long,E/W,speed,course,date,,,mode*checksum
- Example:
\$GPRMC,053740.000,A,2503.6319,N,12136.0099,E,2.69,79.65,100106,,,A*53

3.7 GSM MODEM

Introduction to GSM

GSM (Global System for Mobile Communications) is a digital cellular technology standard used for transmitting voice and data services over mobile networks. Originally operating in the 900 MHz and later 1800 MHz bands in Europe, GSM expanded to include 850 MHz and 1900 MHz bands in the U.S., Australia, and other regions, enabling seamless international roaming across over 218 countries. Its standardized spectrum and coverage (over 80% of the global population) support voice calls, SMS, and low-speed data (up to 9.6 kbps).



Fig 3.17 GSM

Mobile Call Types

- **Mobile-Originated Call (MOC):** Outgoing call from a mobile to a fixed network.
- **Mobile-Terminated Call (MTC):** Incoming call from a fixed network to a mobile device.

History and Evolution

In the 1980s, analog systems like TACS (UK), AMPS (USA), and C-Netz (Germany) were incompatible. To create a unified mobile communication system, CEPT formed the GSM group in 1982. By 1989, the responsibility passed to ETSI, which standardized GSM's digital structure and ISDN compatibility.

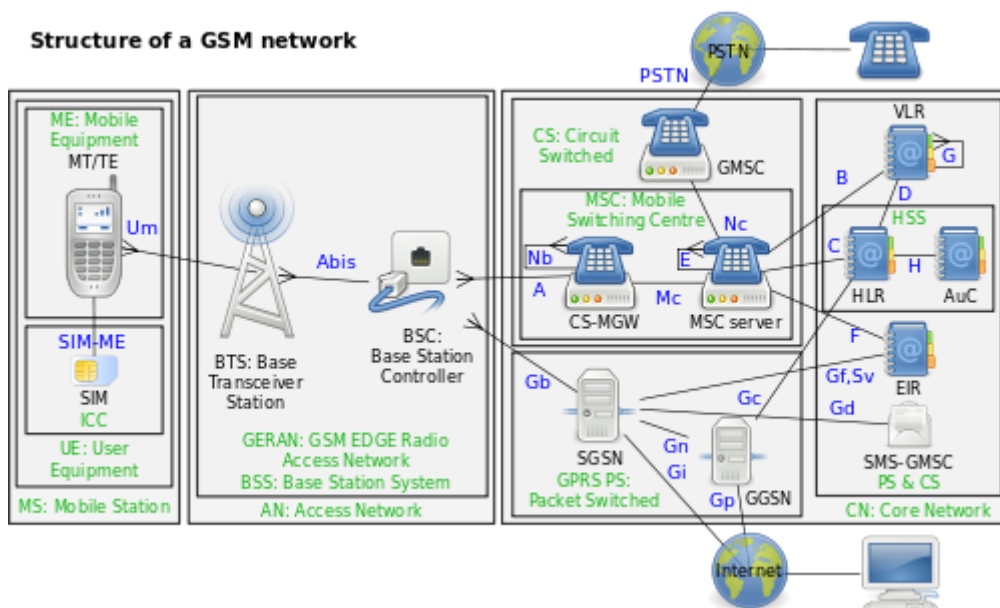


Fig 3.18 Structure of GSM

Architecture & Operation

GSM architecture resembles a computer network with three core components:

1. **Radio Network:** Handles wireless communication and handovers. Composed of Base Transceiver Stations (BTS) and controlled by a Base Station Controller (BSC).
2. **Mobile Switching Network:** Routes calls, manages user info, and facilitates roaming through:
 - **MSC (Mobile Services Switching Center)**
 - **HLR (Home Location Register):** Permanent user data
 - **VLR (Visitor Location Register):** Temporary user data
 - **EIR (Equipment Identity Register):** Validates device identity
3. **Operation & Maintenance Center:** Manages configuration, security, and performance.



Fig 3.19 Working of GSM

Technical Specifications

Parameter	Value
Multiple Access	TDMA / FDMA

Parameter	Value
Frequency Range	890–915 MHz (DL), 933–960 MHz (UL)
Modulation	GMSK
Channel Spacing	200 kHz
Speech Codec	RPE-LTP / 13 kbps
Channel Coding	1/2 rate convolutional
Frame Duration	4.615 ms
Speech Channels / RF	8
Max TX Power	1000 mW

GSM 1800 MHz has a similar structure with improved frequency reuse and reduced interference.

Voice Signal Processing

Voice signals are digitized and compressed using the RPE-LTP codec, reducing bandwidth to 13 kbps. Features like VAD (Voice Activity Detection) and DTX (Discontinuous Transmission) conserve battery by disabling transmission during pauses. Comfort Noise Generators (CNG) replace silence with synthetic background noise. Lost frames are handled by error concealment algorithms using past data.

Security & Ciphering

Authentication Algorithms

- **A3:** Verifies user identity via a 32-bit response (SRES).
- **A8:** Generates a 64-bit session key (K_c).
- **A5/1:** Stream cipher for voice encryption using K_c and a 22-bit frame sequence.

Interfaces in GSM

1. Air Interface (MS ↔ BTS)

- **BCCH:** Broadcasts cell-specific info
- **FCCH/SCH:** Frequency and synchronization
- **RACH/AGCH/NCH:** Access and notification channels
- **SACCH/FACCH/SDCCH:** Control signaling for call setup and maintenance

2. Abis Interface (BTS ↔ BSC)

- Supports traffic and control between BTS and network core.

GSM AT Commands

Used to communicate with a GSM modem via a serial link (default: autobaud, 8N1, RTS/CTS flow). Commands begin with AT and end with carriage return (<CR>).

Basic Commands

Command	Description
AT+CGSN	Read IMEI
A/	Repeat last command
AT+CSQ	Signal quality (RSSI & BER)
AT+CNMI	Configure message indications
AT+CMGR	Read SMS from memory
AT+CMGL	List messages by status
AT+CMGS	Send SMS in text/PDU mode
AT+CMGD	Delete SMS (by index or bulk)

Message Storage Flags in +CMGD

- 0: Delete specific message
- 1-4: Bulk delete by read/sent/unsent/all

CHAPTER 4

CIRCUIT DIAGRAM

4.1 Existing System

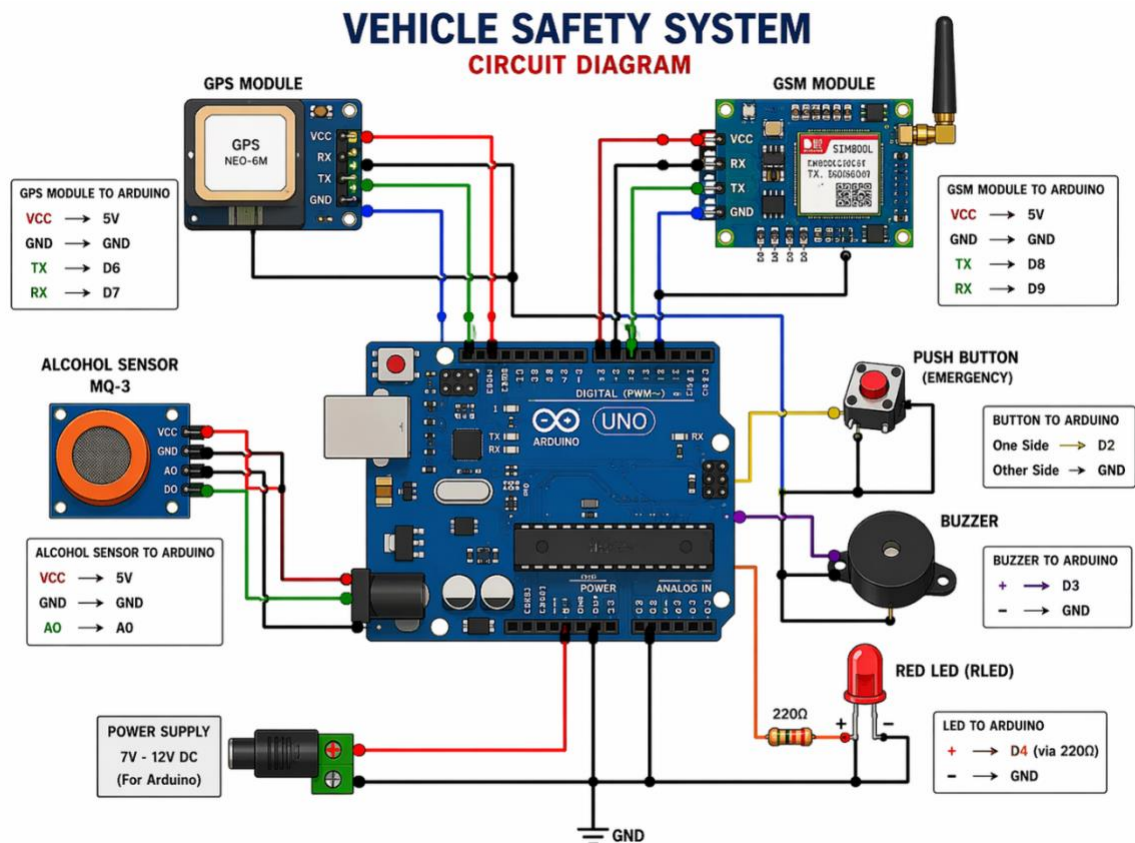


Fig 4.1 Circuit Diagram

4.2 Source Code

```
#include <SoftwareSerial.h>
```

```
#include <TinyGPS++.h>
```

```
// GSM (TX, RX)
```

```
SoftwareSerial gsm(8, 9);
```

```

// GPS (TX, RX)

SoftwareSerial gpsSerial(6, 7);

TinyGPSPlus gps;


// Pins

#define alcoholSensor A0

#define button 2

#define buzzer 3

#define rled 4


int alcoholThreshold = 400;

bool alcoholDetected = false;

bool buttonPressed = false;


String latitude = "";

String longitude = "";


// -----

void setup() {

  Serial.begin(9600);

  gsm.begin(9600);

  gpsSerial.begin(9600);


  pinMode(button, INPUT_PULLUP);

```

```

pinMode(buzzer, OUTPUT);

pinMode(rled, OUTPUT);


digitalWrite(buzzer, LOW);

digitalWrite(rled, LOW);


Serial.println("System Initialized...");
}

// -----

void loop() {

    // Read GPS data

    while (gpsSerial.available()) {

        gps.encode(gpsSerial.read());

    }

    if (gps.location.isValid()) {

        latitude = String(gps.location.lat(), 6);

        longitude = String(gps.location.lng(), 6);

    }

    // Read Alcohol Sensor

    int alcoholValue = analogRead(alcoholSensor);

```



```

Serial.print("Alcohol Value: ");

Serial.println(alcoholValue);


// ----- Alcohol Detection -----

if (alcoholValue > alcoholThreshold && !alcoholDetected) {

    alcoholDetected = true;


    digitalWrite(buzzer, HIGH);

    digitalWrite(rled, HIGH);


    sendSMS("ALERT! Alcohol detected!\nLocation:\nLat:" + latitude + "\nLng:" +
longitude);

}


if (alcoholValue < alcoholThreshold) {

    alcoholDetected = false;

    digitalWrite(buzzer, LOW);

    digitalWrite(rled, LOW);

}


// ----- Emergency Button -----

if (digitalRead(button) == LOW && !buttonPressed) {

    buttonPressed = true;


    digitalWrite(buzzer, HIGH);

```

```

digitalWrite(rled, HIGH);

    sendSMS("EMERGENCY! Button Pressed!\nLocation:\nLat:" + latitude + "\nLng:" +
longitude);
}

if (digitalRead(button) == HIGH) {
    buttonPressed = false;
}

delay(1000);
}

// -----

void sendSMS(String message) {
    Serial.println("Sending SMS...");

    gsm.println("AT");
    delay(1000);

    gsm.println("AT+CMGF=1"); // Text mode
    delay(1000);

    gsm.println("AT+CMGS=\"" + 91XXXXXXXXXXXX "\""); // Replace with your number
    delay(1000);

```

```
gsm.print(message);  
  
delay(500);  
  
gsm.write(26); // CTRL+Z to send  
  
delay(5000);  
  
Serial.println("SMS Sent");  
}
```

4.3 Flowchart

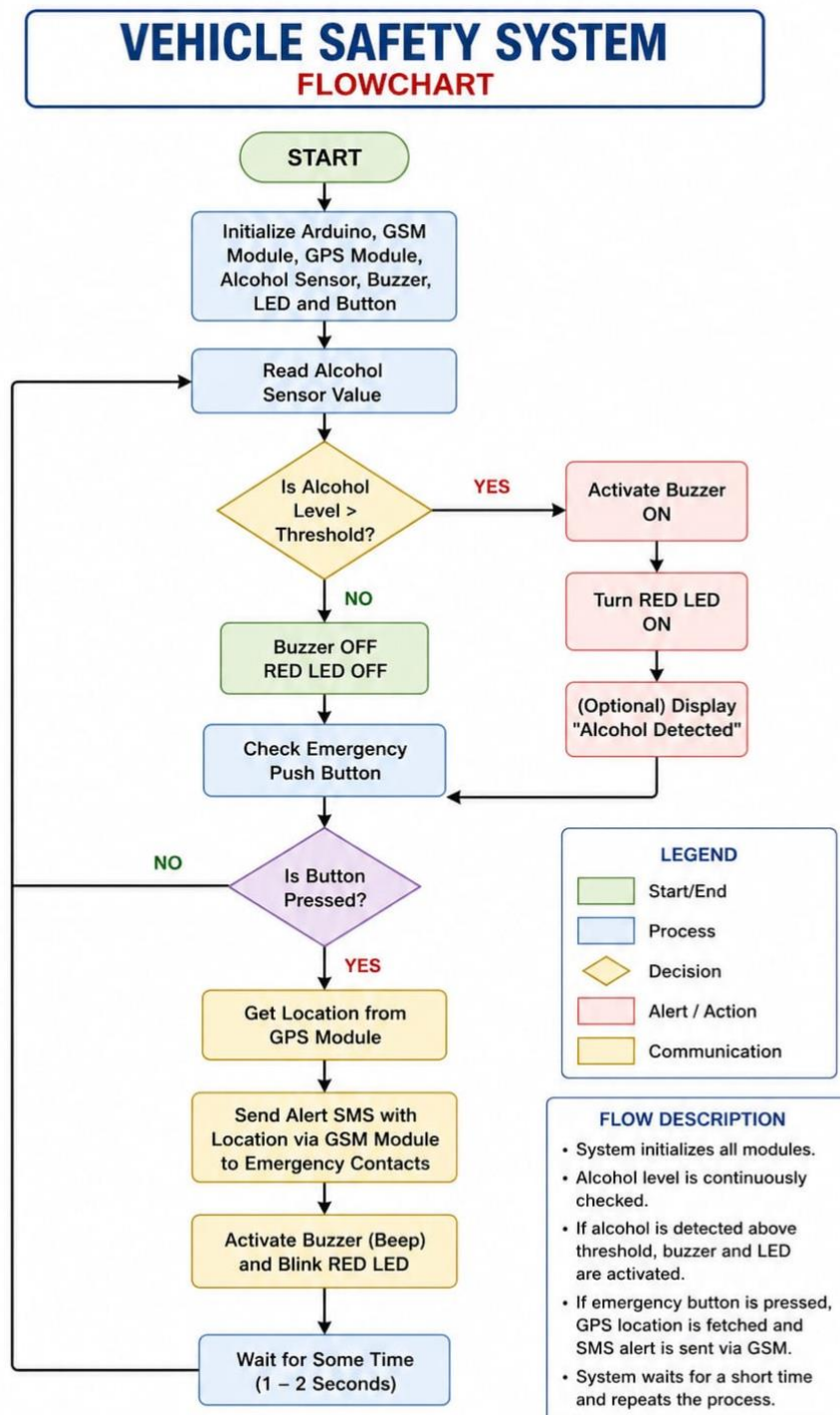


Fig 4.2 Flowchart

CHAPTER 5

SOFTWARE DEVELOPMENT

5.1 Software Development

These are the steps you need to follow in order to be up and running:

1. Get an Arduino board
2. Download the Arduino environment
3. Install the USB drivers
4. Connect the board
5. Upload a program

Get an Arduino board

The Arduino i/o board is a simple circuit featuring the ATmega8 processor from Atmel. The board is composed of a printed circuit board (PCB) and electronic parts.

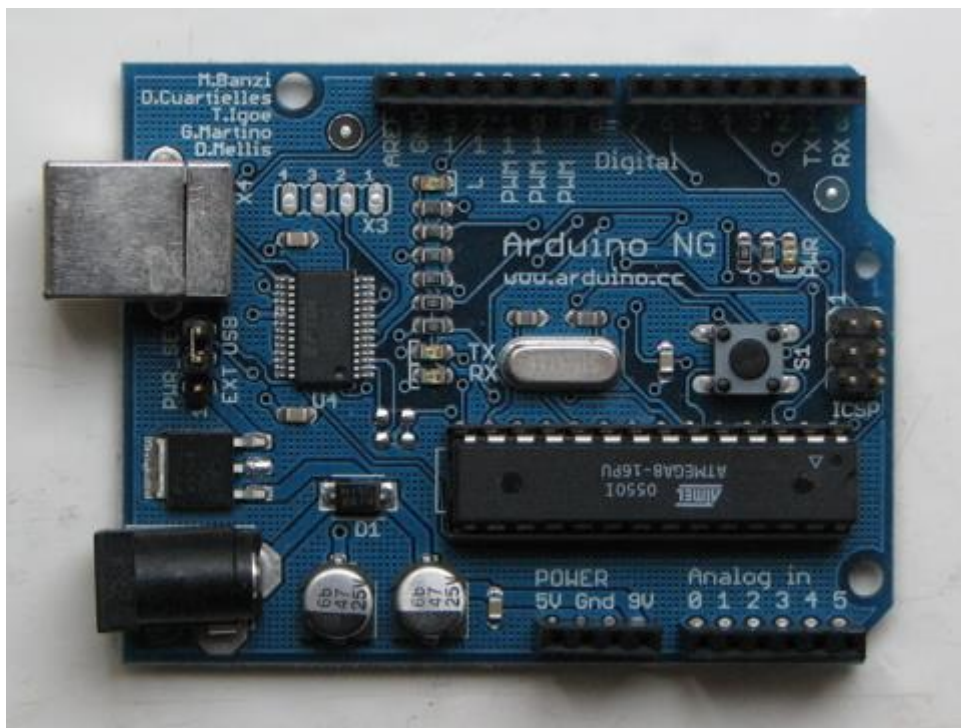


Fig 5.1 Arduino Board

There are a few ways to get an Arduino board:

buy a ready made board. See how you can buy a board or just the PCB.

- European distributor
- US distributor

Build your own board. If you want you can build your own PCB just by downloading the CAD files from the Hardware page. Extract the .brd file and send it to a PCB manufacturer. Be aware that manufacturing a single pcb will be very expensive. It's better to get together with other people and make 20 or 30 at a time. Since you get the full CAD files you can make your own customised version of Arduino. if you make modifications or fix bugs please send us your changes!

Purchase parts.

Purchase the parts from any electronics store. The Serial version in particular has been designed to use the most basic parts that can be found anywhere in the world. The USB version on the other hand requires some advanced soldering skills because of the FTDI chip that is an smd part.

Assemble the board.

We put together a step by step guide on how to build an arduino board.

Program the boot loader.

In order for the development environment to be able to program the chip, this has to be programmed with a piece of code called *bootloader*. See the bootloader page on how to program it on your chip.

Download the ARDUINO environment

To program the Arduino board you need the Arduino environment.

Download Arduino: From the software page.

Linux note: For help getting the Arduino IDE running on Debian, please see the FAQ ("How can I run the Arduino IDE under Linux?").

Mac OS X note: After downloading the IDE, run the macosx_setup.command. It corrects permission on a few files for use with the serial port and will prompt you for your password. You may need to reboot after running this script.

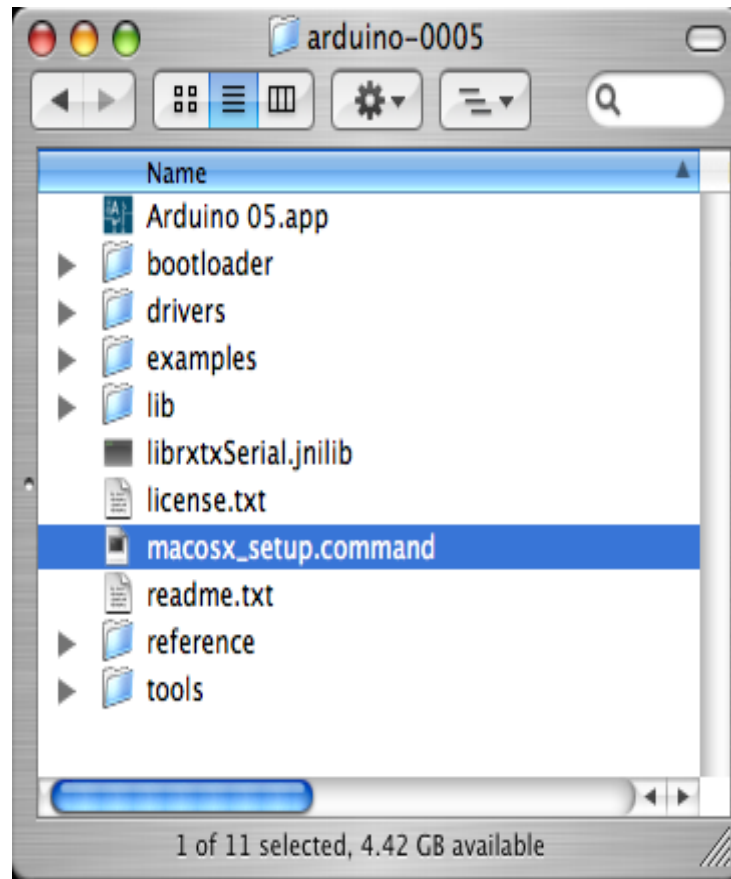


Fig 5.2 Arduino Configuration

Install the USB drivers

If you are using a USB Arduino, you will need to install the drivers for the FTDI chip on the board.

These can be found in the drivers directory of the Arduino distribution.

On Windows, you will need to unzip FTDI USB Drivers.zip. Then, when you plug in the Arduino board, point the Windows *Add Hardware* wizard to the FTDI USB Drivers directory.

On the Mac, mount the FTDIUSBSerialDriver_v2_1_6.dmg (on PPC machines) or the FTDIUSBSerialDriver_v2_2_6_Intel.dmg (on Intel machines) disk image and run the included FTDIUSBSerialDriver.pkg.

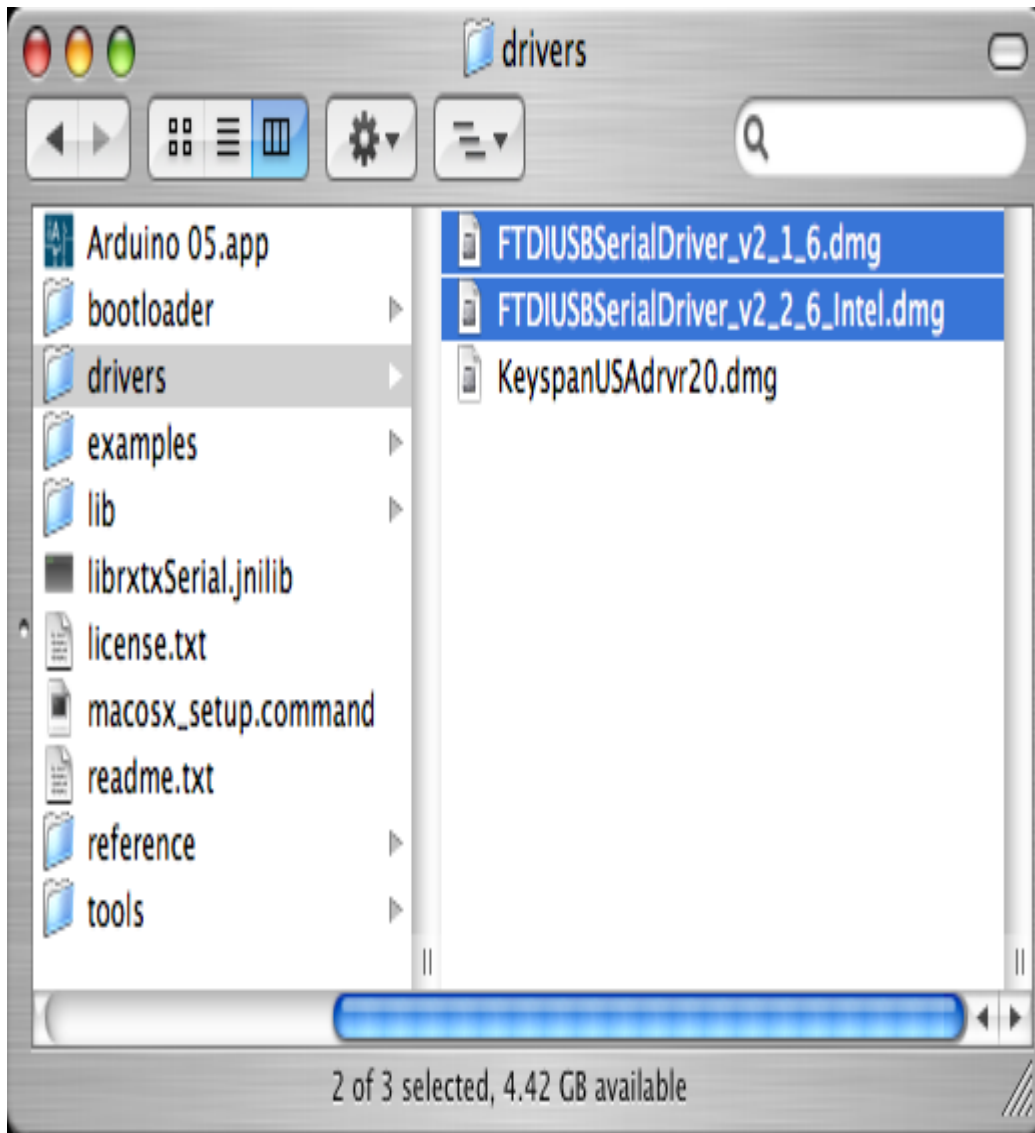


Fig 5.3 Drive Configuration

The latest version of the drivers can be found on the [FTDI website](#).

Connect the board

If you're using a serial board, power the board with an external power supply (6 to 25 volts DC, with the core of the connector positive). Connect the board to a serial port on your computer.

On the USB boards, the power source is selected by the jumper between the USB and power plugs. To power the board from the USB port (good for controlling low power devices like LEDs), place the jumper on the two pins closest to the USB plug. To power the board from an external power supply (needed for motors and other high current devices), place the jumper on the two pins closest to the power plug. Either way, connect the board to a

USB port on your computer. On Windows, the Add New Hardware wizard will open; tell it you want to specify the location to search for drivers and point to the folder containing the USB drivers you unzipped in the previous step.

The power LED should go on.

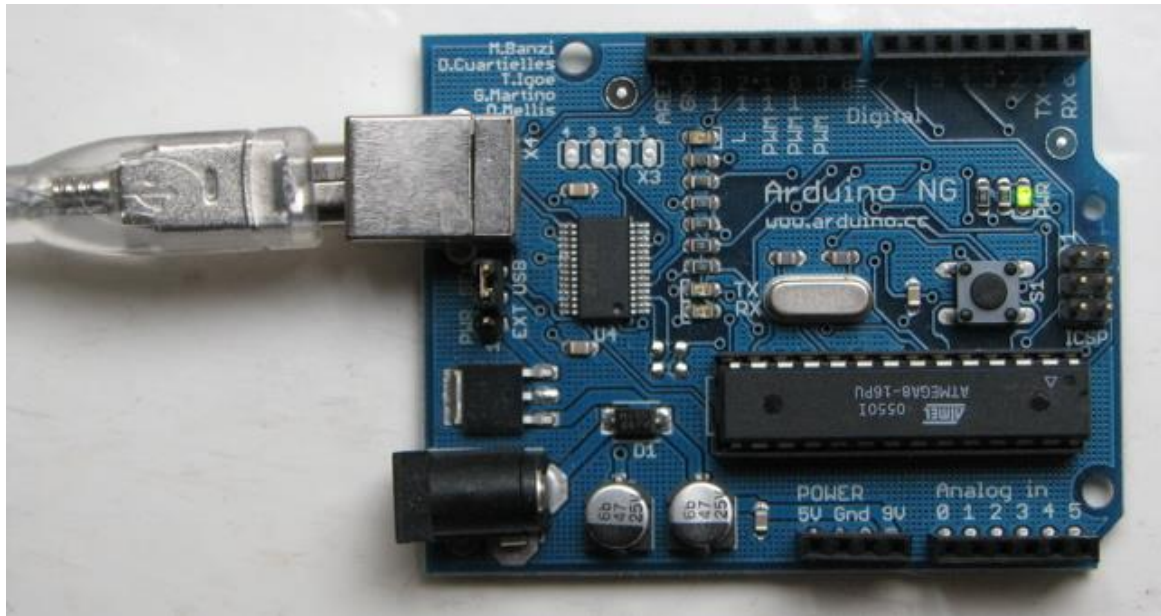


Fig 5.4 Arduino board connection

Upload a program

Open the LED blink example sketch: File > Sketchbook > Examples > led_blink.

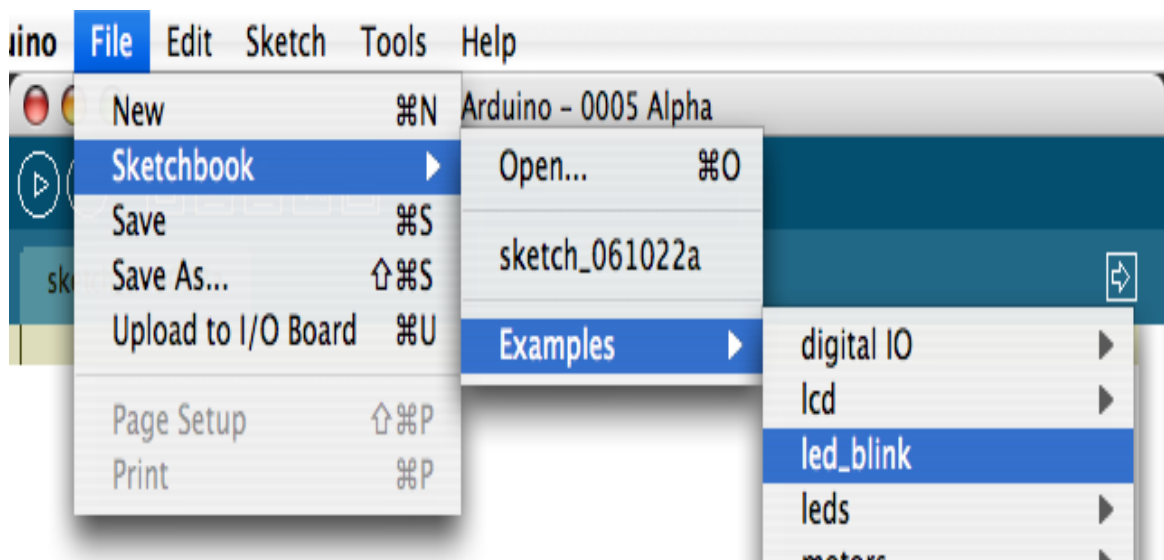
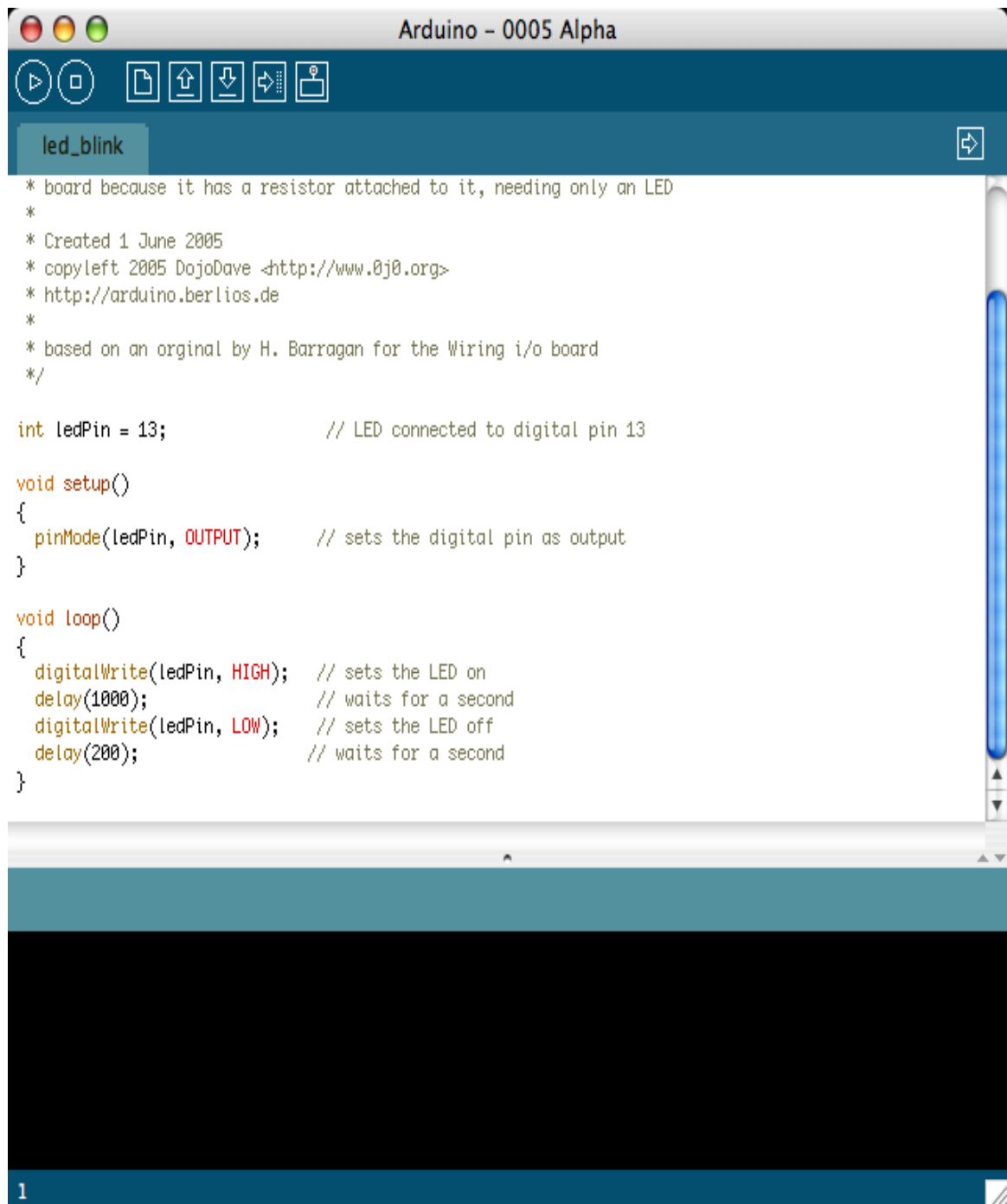


Fig 5.5 Uploading program from file

Here's what the code for the LED blink example looks like.



```
Arduino - 0005 Alpha

led_blink

* board because it has a resistor attached to it, needing only an LED
*
* Created 1 June 2005
* copyleft 2005 DojoDave <http://www.0j0.org>
* http://arduino.berlios.de
*
* based on an original by H. Barragan for the Wiring i/o board
*/

int ledPin = 13;           // LED connected to digital pin 13

void setup()
{
  pinMode(ledPin, OUTPUT); // sets the digital pin as output
}

void loop()
{
  digitalWrite(ledPin, HIGH); // sets the LED on
  delay(1000);                // waits for a second
  digitalWrite(ledPin, LOW);  // sets the LED off
  delay(200);                 // waits for a second
}
```

Fig 5.6 Code for Arduino

Select the serial device of the Arduino board from the Tools | Serial Port menu. On Windows, this should be COM1 or COM2 for a serial Arduino board, or COM3, COM4, or COM5 for a USB board. On the Mac, this should be something like /dev/cu.usbserial-1B1 for a USB board, or something like /dev/cu.USA19QW1b1P1.1 if using a Keyspan adapter with a serial board (other USB-to-serial adapters use different names).

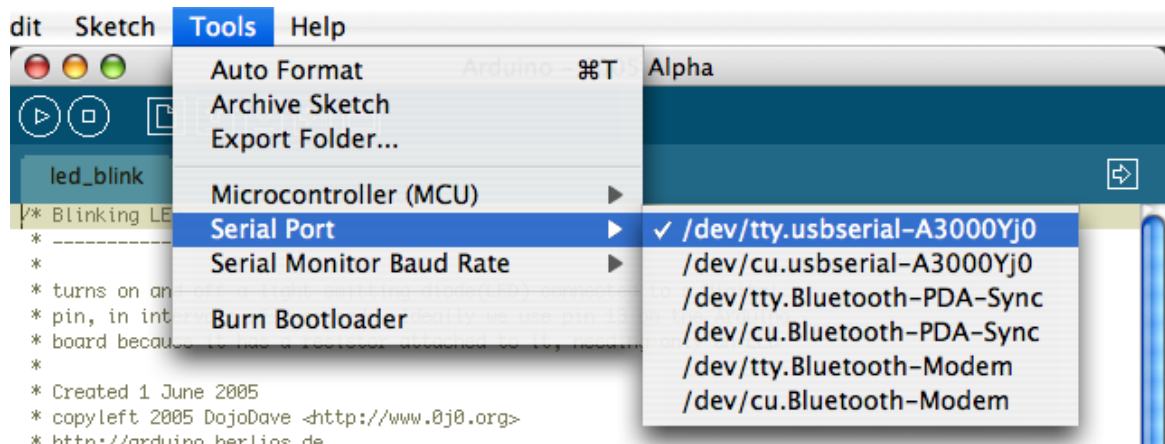
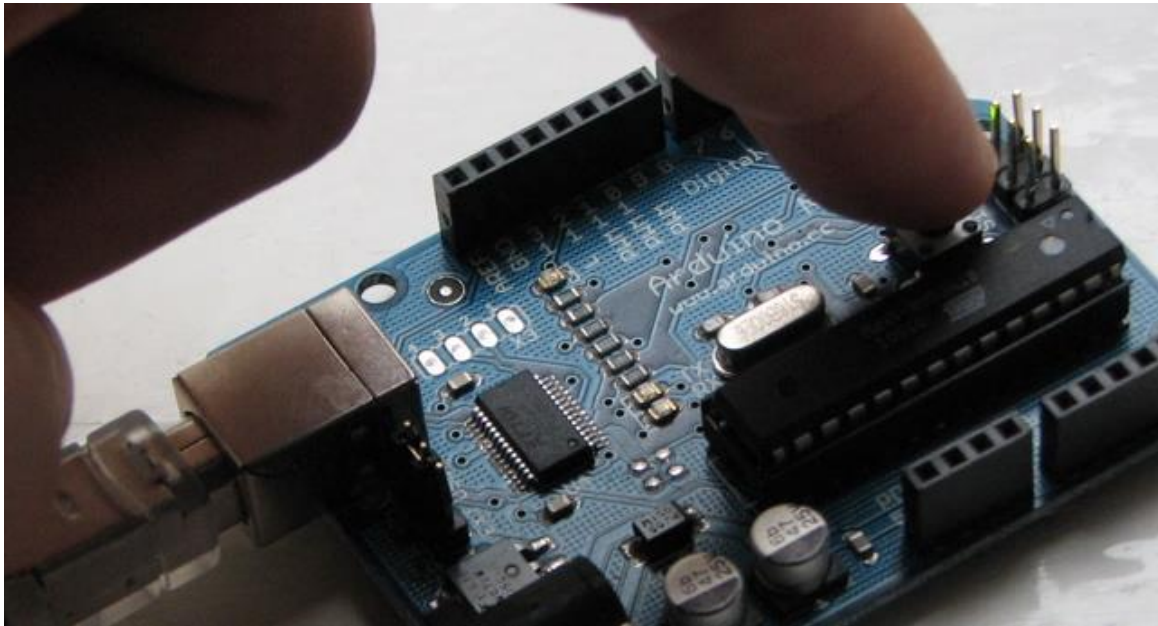


Fig 5.7 Applying tools

Push the reset button on the board then click the *Upload* button in the IDE. Wait a few seconds. If successful, the message "Done uploading." will appear in the status bar.



If the Arduino board doesn't show up in the Tools | Serial Port menu, or you get an error while uploading, please see the [FAQ](#) for troubleshooting suggestions.

A few seconds after the upload finishes, you should see the amber (yellow) LED on the board start to blink.

Example Programs

To open the blink sketch, you will need to go to

File

> **Examples**

> **Basics**

> **Blink**

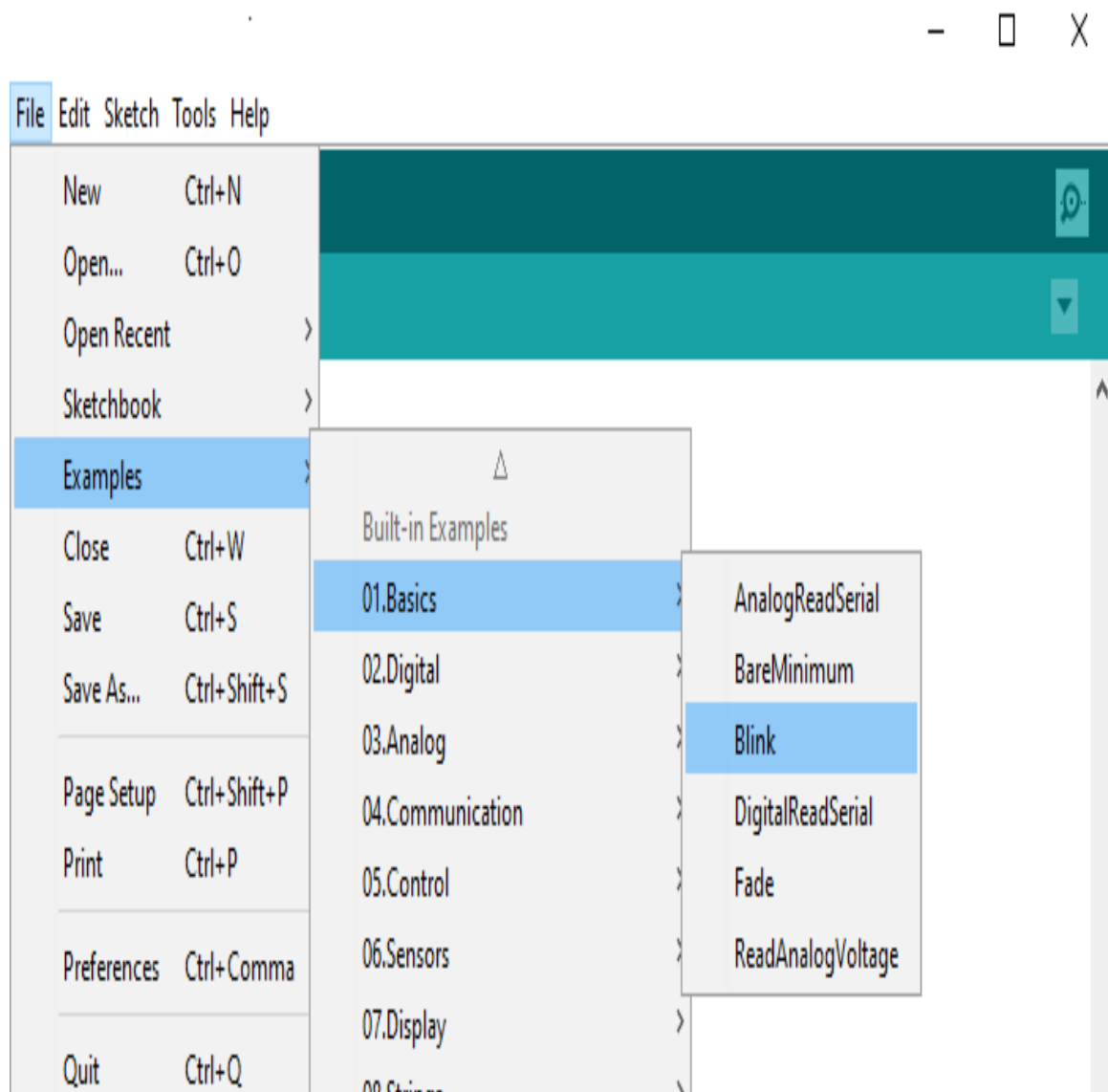


Fig 5.8 Adding examples



Fig 5.9 Arduino console - 1

Now you should have a fully coded blink sketch that looks like the image below.

Next, you need to click on the verify button (check mark) that's located in the top left of the IDE box. This will compile the sketch and look for errors. Once it says "Done Compiling"

you are ready to upload it. Click the upload button (forward arrow) to send the program to the Arduino board.



Fig 5.10 Arduino console - 2

The built-in LEDs on the Arduino board will flash rapidly for a few seconds and then the program will execute. If everything went correctly, the LED on the breadboard should turn on for a second and then off for a second and continue in a loop.

Change The Code

Before we go to the next project, let's change some of the code in the “Blink” sketch to make it do something different. Playing around with the sketch will help you start to learn how the code controls the board.

Keep the Arduino board connected and change the delay portion of the code from (1000) to (200). Click the verify button on the top left of the IDE and then click upload. This should make the LED on the breadboard blink faster.

NOTE – Arduino measures time in milliseconds and 1000 milliseconds = 1 second. The original code (1000) turns on the LED for 1 second and then off for 1 second. By adjusting the code from (1000) to (200) it shortens the time between on and off which makes it blink faster.


```

// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(200);
  digitalWrite(LED_BUILTIN, LOW);
  delay(200);
}

```

Changed to 200

Changed to 200

Fig 5.11 Arduino console -3

Arduino Project 2: LED w/ Switch

Now it's time to talk switches and how they can be incorporated into Arduino projects. A switch is a electrical component that completes a circuit when pushed and breaks the circuit when released. In this project, we will be using a small pushbutton switch to control an LED.

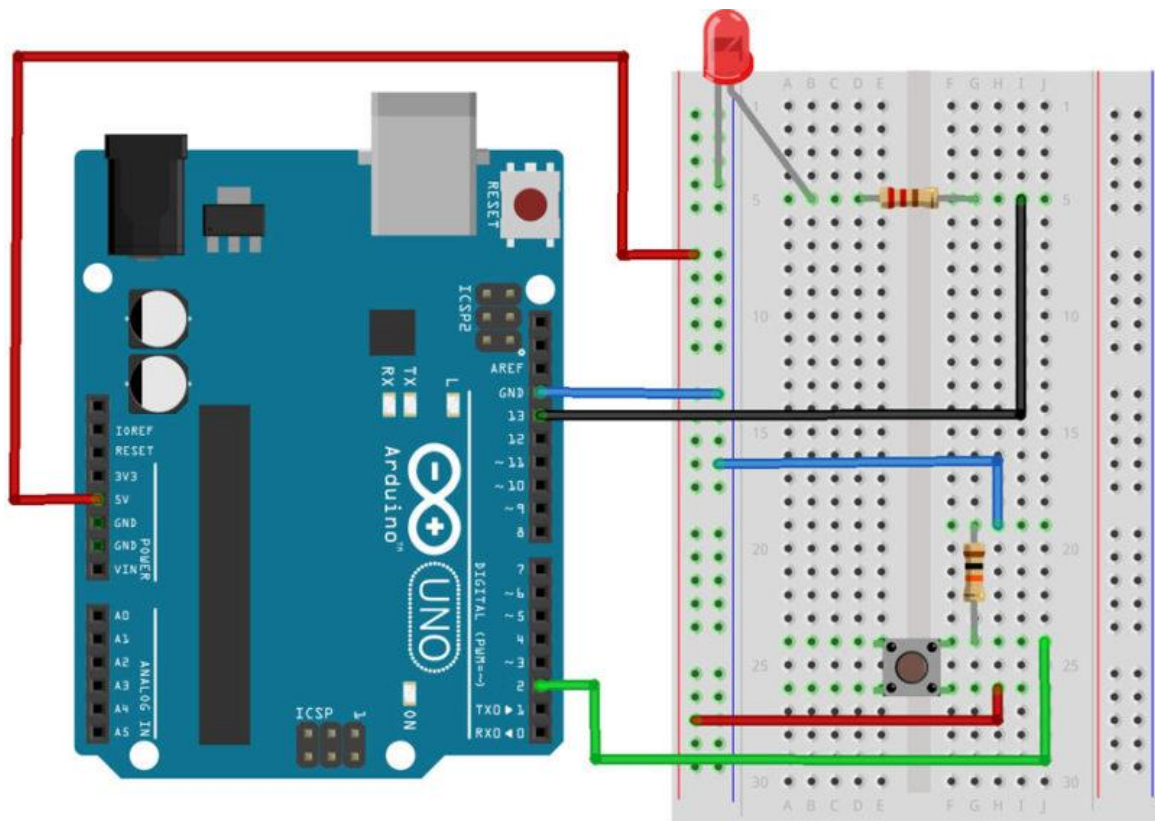


Fig 5.11 Arduino Motherboard

Required Parts

- Arduino Uno Board
- Breadboard – half size
- Jumper Wires
- USB Cable
- LED (5mm)
- Push button switch
- 10k Ohm Resistor
- 220 Ohm Resistor

Connect The Parts

You can build your Arduino circuit by looking at the breadboard image above or by using the written description below. In the written description, we will use a letter/number combo that refers to the location of the component. If we mention H19 for example, that refers to column H, row 19 on the breadboard.

Step 1 – Connect the blue jumper wire from the GND on the Arduino to the GND rail (blue line) on the breadboard near A13

Step 2 – Connect the blue jumper wire from the GND rail on the breadboard near A17 to H19

Step 3 – Connect the red jumper wire from the power rail on the breadboard around row A27 to H26

Step 4 – Connect the green jumper wire from pin 2 on Arduino to J24 on the breadboard

Step 5 – Place one leg of a 10k Ohm resistor in G19 and the other leg in G24

Step 6 – Place the pushbutton switch into F24, F26, E24 and E26

Step 7 – Place one leg of a 220 Ohm resistor in D5 and the other leg in G5

Step 8 – Insert the short leg of the LED in the GND rail around A5 and the long leg in B5

Step 9 – Connect the black jumper wire from pin 13 on the Arduino to I5 on the breadboard

Step 10 – Connect the red jumper wire from 5V on the Arduino to power rail (+) near A8

Step 11 – Connect the Arduino Uno to your computer via USB cable

Upload The Switch Sketch

Now it's time to upload the sketch to the Arduino that will allow us to use a switch. As with the blink sketch, there are example programs already loaded in the Arduino IDE that we will be using.

In order to use a switch, we have to load the file called "Button" which can be found here: **File**

> **Examples**

> **Digital**

> **Button**

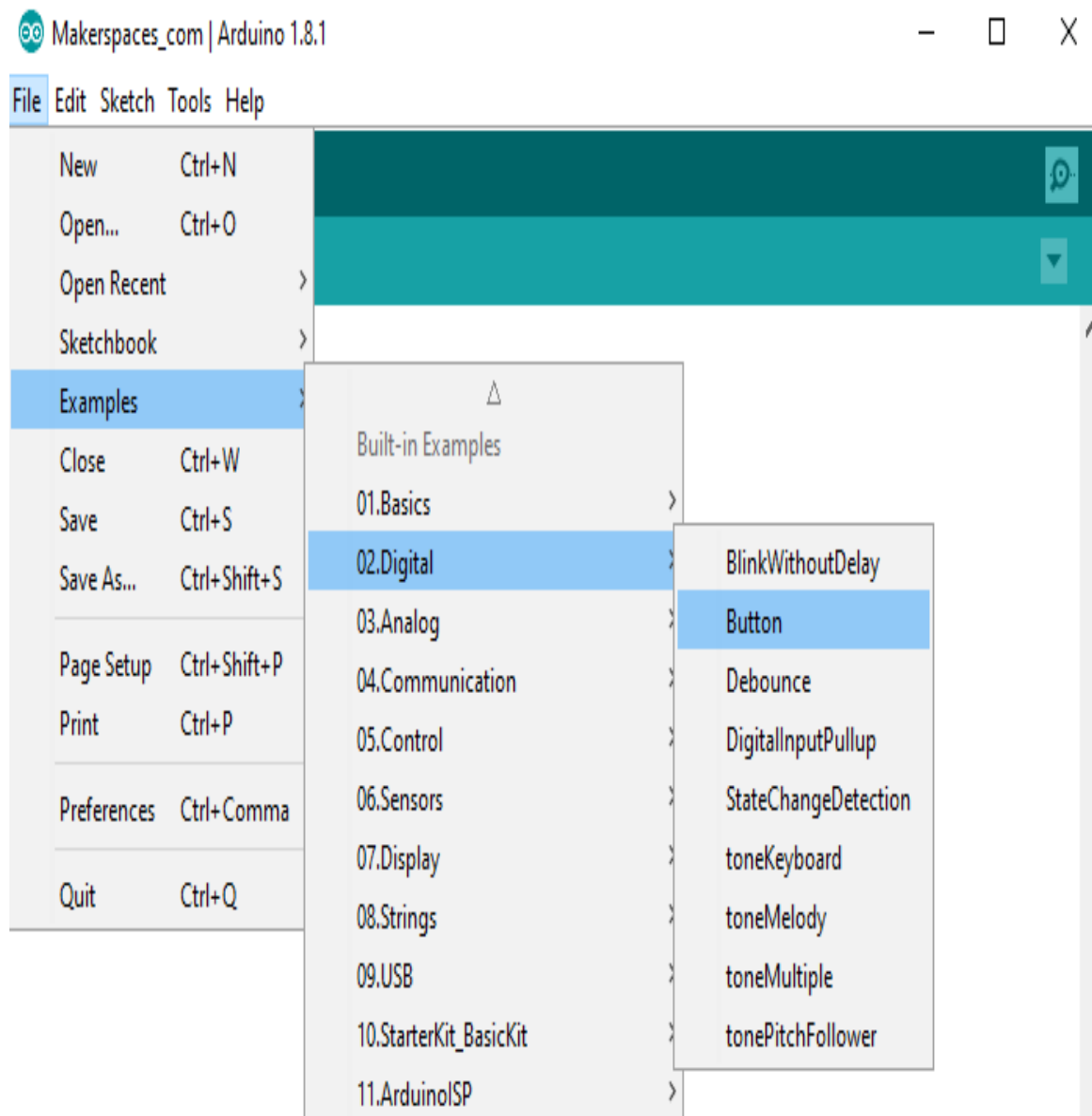


Fig 5.12 Arduino file selection

Now you should have a fully coded button sketch that looks like the image below.

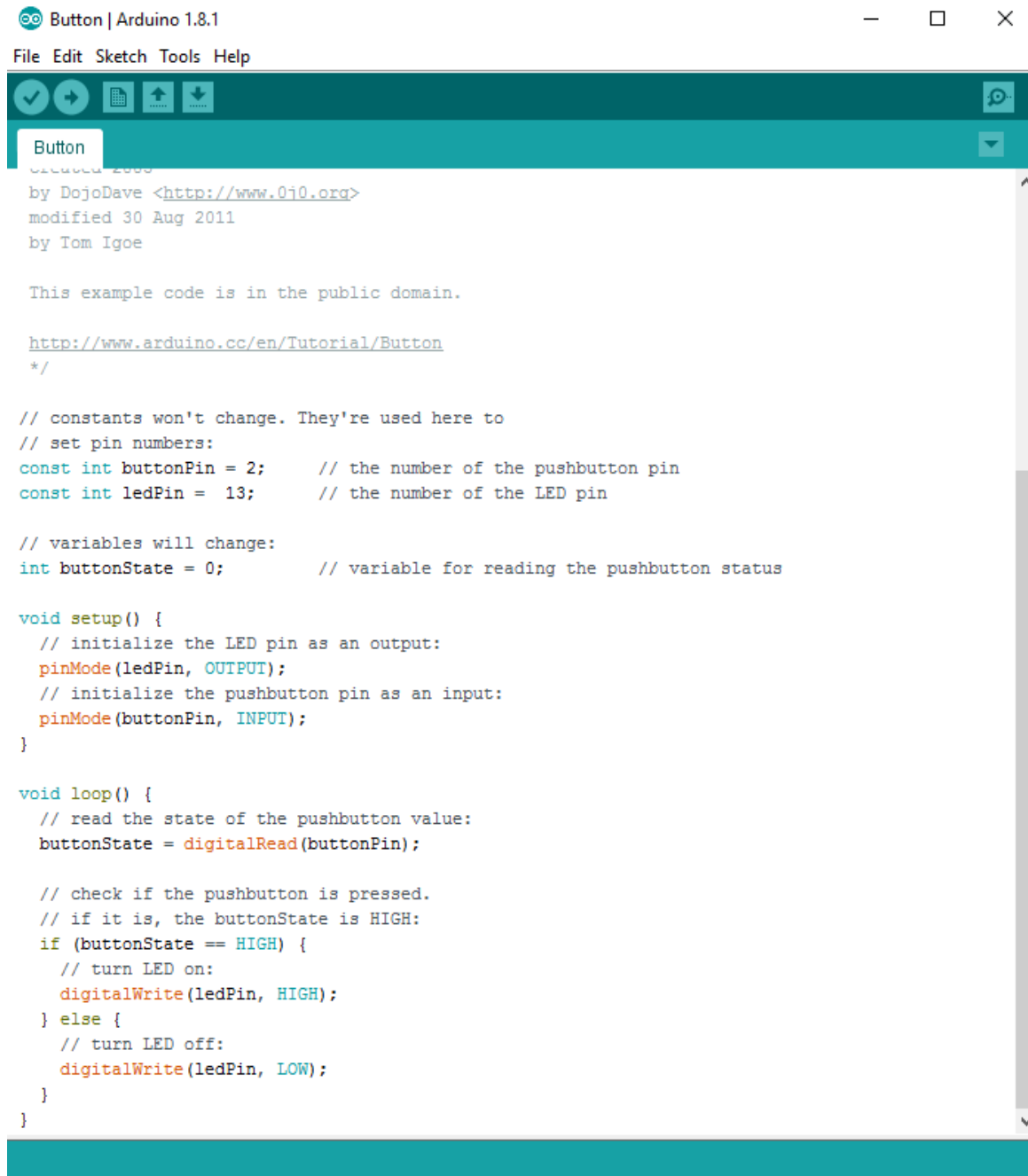


Fig 5.13 Arduino console -4

Next, you need to click on the verify button (check mark) that's located in the top left of the IDE box. Once it says "Done Compiling" you are ready to upload it. Click the upload button (forward arrow) to send the program to the Arduino board.

CHAPTER 6

OUTPUT IMAGES

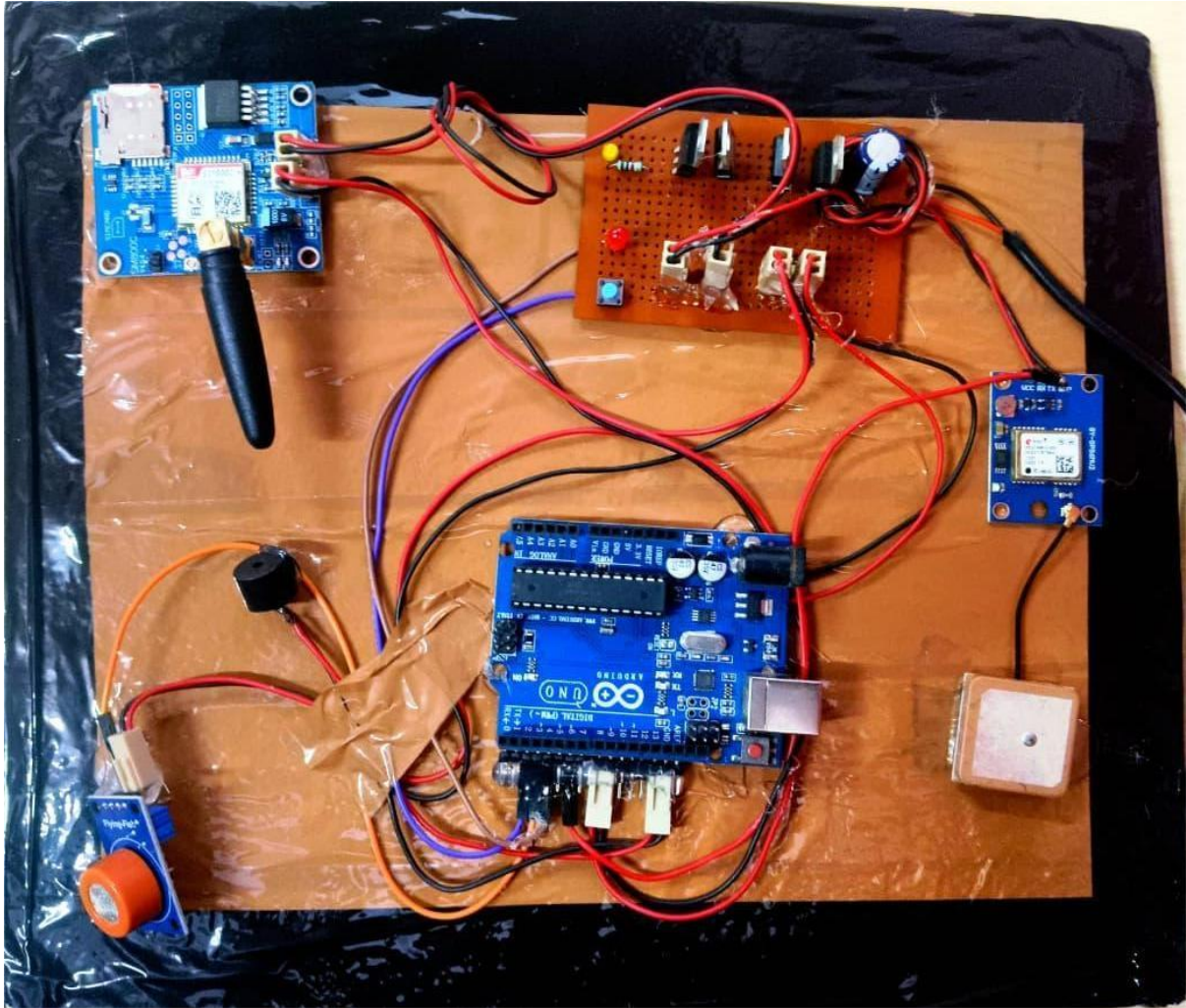


Fig 6.1 Before the Alert

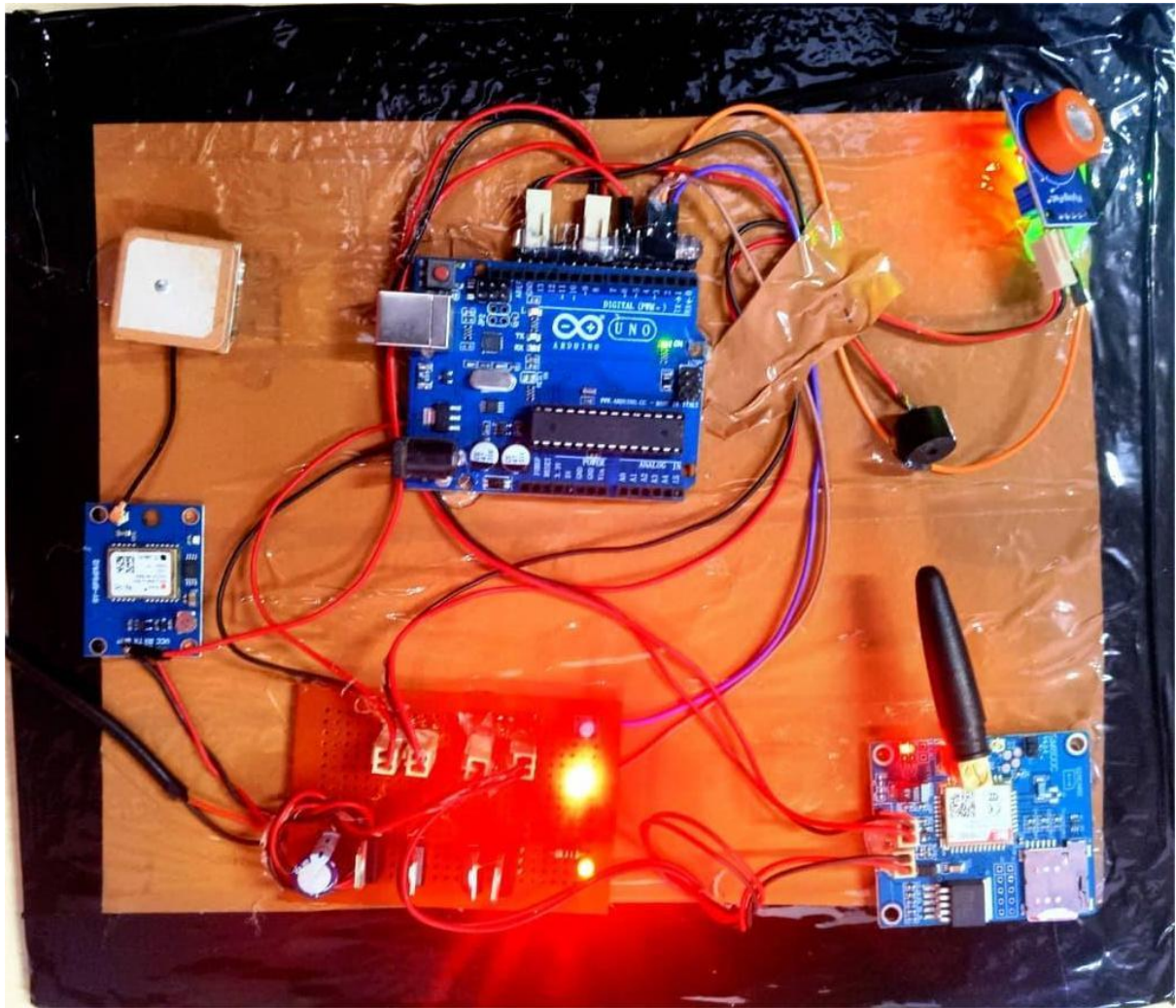


Fig 6.2 After the alert

CHAPTER 7

CONCLUSION

The proposed Vehicle Safety System using Arduino UNO with GSM and GPS successfully demonstrates an effective solution to enhance road safety by combining prevention and emergency response mechanisms. The system integrates key components such as an alcohol sensor, GPS module, GSM module, push button, buzzer, and LED to create a comprehensive safety framework.

The alcohol detection feature plays a crucial role in preventing accidents caused by drunk driving by alerting the driver and indicating unsafe conditions through audio-visual signals. The inclusion of GPS and GSM modules enables real-time location tracking and instant communication, ensuring that emergency alerts are sent promptly to predefined contacts. This significantly reduces response time during critical situations.

Additionally, the emergency push button provides a manual way to seek help, making the system reliable even in unpredictable scenarios. The use of the Arduino UNO makes the system cost-effective, easy to implement, and scalable for further enhancements.

Overall, the system offers a **proactive and intelligent approach** to vehicle safety by:

- Preventing accidents through alcohol detection
- Providing real-time tracking and communication
- Enabling quick emergency response
- Ensuring low-cost and practical implementation

This project can be further extended with IoT integration, mobile applications, and AI-based monitoring to create a more advanced and fully automated vehicle safety ecosystem.

CHAPTER 8

REFERENCES

1. Arduino Official Documentation – <https://www.arduino.cc>
2. GSM Module (SIM800/900) AT Command Manual
3. GPS Module NEO-6M Datasheet
4. MQ-3 Alcohol Sensor Datasheet
5. Embedded Systems Design – Raj Kamal
6. “IoT Based Vehicle Tracking System” – IEEE Papers
7. “Accident Detection and Alert System” – International Journal of Engineering Research
8. “Smart Vehicle Safety System using Arduino” – IJERT
9. “Alcohol Detection System in Vehicles” – IEEE Conference Papers
10. Programming Arduino: Getting Started with Sketches – Simon Monk
11. Electronics For You Magazine (EFY)
12. National Highway Traffic Safety Reports on Drunk Driving
13. GSM Based Vehicle Tracking System – ResearchGate Publications
14. GPS Technology and Applications – Kaplan & Hegarty
15. Microcontroller and Embedded Systems – Muhammad Ali Mazidi
16. IoT Fundamentals – David Hanes
17. AT Commands Reference Guide for SIMCOM Modules
18. Sensors and Transducers – D. Patranabis
19. Arduino Projects Book – Arduino.cc
20. Road Safety Annual Reports – Government Transport Authority